# GPU 内存层次结构

## GPU Memory Hierarchy

原文作者：@techNmak。

先列专业术语对照表，再提供逐页完整中文译文。每个原文页面的译文之后附原文矢量对照页，完整保留公式、图表、代码和英文引用，便于核对。

正文中文：LXGW WenKai（霞鹜文楷）；英文与数字：Times New Roman。原文对照页保留原始字体。

本文采用翻译辅助服务生成初稿，统一技术术语并检查结构、字体和导航；涉及精确数值、公式或实现细节时，可直接对照相邻原文页。

# 专业术语翻译对照表

| 英文原词 | 中文译名 | 缩写／符号 |
|---|---|---|
| GPU Memory Hierarchy | GPU 内存层次结构 | |
| Shared Memory | 共享内存 | |
| Data Movement | 数据移动 | |
| Coalescing | 合并访问 | |
| Registers | 寄存器 | |
| Kernels | 内核 | |
| Tiling | 分块 | |
| Arithmetic Intensity | 算术强度 | |
| Memory Hierarchy | 内存层次结构 | |
| Synchronization | 同步 | |
| Global Memory | 全局内存 | |
| Memory Access | 内存访问 | |
| Device Memory | 设备内存 | |
| Mental Model | 理解框架 | |
| Performance | 性能 | |
| Throughput | 吞吐量 | |
| Occupancy | 占用率 | |
| Profiling | 性能剖析 | |
| Register | 寄存器 | |
| L2 Cache | 二级缓存 | |
| Latency | 延迟 | |
| Tensor | 张量 | |
| Kernel | 内核 | |
| Core | 核心 | |
| SM | 流式多处理器 | SM |
| Local Memory | 局部内存 | |
| Attention | 注意力机制 | Attention |
| Warp | 线程束 | |
| GEMM | 通用矩阵乘法 | GEMM |
| FlashAttention | FlashAttention（方法名保留） | |
| Activations | 激活 | |
| Vocabulary | 词表 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Generation | 代际 | |
| Inference | 推理 | |
| Training | 训练 | |
| Prefill | 预填充 | |
| Activation | 激活 | |
| Values | 值 | |
| Models | 模型 | |
| Value | 值 | V |
| Model | 模型 | |
| Register Pressure | 寄存器压力 | |
| Texture Cache | 纹理缓存 | |
| Dot Product | 点积 | |
| Matrix | 矩阵 | |
| Memory Traffic | 内存流量 | |
| Register File | 寄存器文件 | |
| Residency | 驻留状态 | |
| Spilling | 溢出 | |
| Resident | 驻留 | |
| Warps | 线程束 | |
| Coalesced Access | 合并访问 | |
| GPU Generation | GPU 代际 | |
| Fused Kernels | 融合内核 | |
| Thread Block | 线程块 | |
| Working Set | 工作集 | |
| Memory Bandwidth | 内存带宽 | |
| Memory Footprint | 内存占用 | |
| Asynchronous Copy | 异步复制 | |
| Cache Hit | 缓存命中 | |
| Operators | 算子 | |
| Profiler | 性能剖析器 | |
| Routing | 路由 | |
| Cache Capacity | 缓存容量 | |
| Normalization | 归一化 | |
| Cache Line | 缓存行 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Tokens | 词元 | |
| Tensor Shape | 张量形状 | |
| Wall-clock Time | 实际耗时 | |
| Floating-point | 浮点 | |
| FLOPs | 浮点运算次数 | FLOPs |
| Matrix Multiplication | 矩阵乘法 | |
| Tensor Cores | 张量核心 | |
| Matrices | 矩阵 | |
| Arithmetic Work | 算术工作量 | |
| Bandwidth-bound | 带宽受限 | |
| Launch Overhead | 启动开销 | |
| Roofline Model | 屋顶线模型 | |
| Compute-bound | 计算受限 | |
| FLOP Count | 浮点运算次数 | |
| Precision | 精度 | |
| Vector | 向量 | |
| FP32 | 32 位浮点 | FP32 |
| FLOP | 浮点运算 | |
| Scaling | 缩放 | |
| Dense Attention | 稠密注意力 | |
| Recomputation | 重计算 | |
| Score Matrix | 分数矩阵 | |
| Operator | 算子 | |
| Softmax | Softmax 归一化 | Softmax |
| Combine | 合并 | |
| Head Dimension | 头维度 | |
| Memory-bound | 内存受限 | |
| KV Cache | 键值缓存 | KV Cache |
| Query | 查询 | Q |
| Token | 词元 | |
| Keys | 键 | |
| Continuous Batching | 连续批处理 | |
| Forward Pass | 前向传播 | |
| Batch Size | 批大小 | |

| 英文原词 | 中文译名 | 缩写／符号 |
| --- | --- | --- |
| Gradient | 梯度 | |
| Fused Kernel | 融合内核 | |
| Tensor Core | 张量核心 | |
| High-bandwidth Memory | 高带宽内存 | HBM |
| Pipeline | 流水线 | |
| GPU Generations | GPU 代际 | |
| Wavefront | 波前 | |
| Serving | 推理服务 | |
| Exact Attention | 精确注意力 | |
| Index | 索引 | |

# 目录导航

点击章节标题跳转到中文译文，点击“原文”核对公式和图表。页码按本译本最终分页生成。


封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

GPU 内存层次结构（GPU Memory Hierarchy） · 第 8 页 ［原文 1］

如何阅读本手册 · 第 10 页 ［原文 2］

内容 · 第 12 页 ［原文 3］

内容续 · 第 14 页 ［原文 4］

1 为什么 GPU 需要 内存层次结构（Memory Hierarchy） · 第 16 页 ［原文 5］

2 层次结构一览 · 第 18 页 ［原文 6］

3 容量、流量、带宽和 延迟（Latency） · 第 20 页 ［原文 7］

4 寄存器（Registers）：最接近的工作状态 · 第 22 页 ［原文 8］

5 寄存器（Register） 文件是有限资源 · 第 24 页 ［原文 9］

6 局部内存（Local Memory） 和 寄存器（Register） 泄漏 · 第 26 页 ［原文 10］

7 共享内存（Shared Memory）：显式片上存储 · 第 28 页 ［原文 11］

8 共享内存（Shared Memory） 不仅仅是另一个缓存 · 第 30 页 ［原文 12］

9 L1：小型、依赖于架构的第一级缓存 · 第 32 页 ［原文 13］

10 L2：HBM 之前的设备范围内重用 · 第 34 页 ［原文 14］

11 HBM 和 设备内存（Device Memory） · 第 36 页 ［原文 15］

12 HBM 不是 全局内存（Global Memory） 的同义词 · 第 38 页 ［原文 16］

13 负载在层次结构中的旅程 · 第 40 页 ［原文 17］

14 时间局部性 · 第 42 页 ［原文 18］

15 空间局部性 · 第 44 页 ［原文 19］

16 工作组和复用距离 · 第 46 页 ［原文 20］

17 缓存命中、未命中和下游流量 · 第 48 页 ［原文 21］

18 合并访问（Coalescing）：组织 线程束（Warp） 的内存请求 · 第 50 页 ［原文 22］

19 工作示例：连续负载和跨步负载 · 第 52 页 ［原文 23］

20 分块（Tiling） 将容量转化为再利用 · 第 54 页 ［原文 24］

21 通用矩阵乘法（GEMM） 也是一个内存重用问题 · 第 56 页 ［原文 25］

移动 22 字节与执行算术运算 · 第 58 页 ［原文 26］

23 为什么元素 内核（Kernels） 和 通用矩阵乘法（GEMM） 的行为不同 · 第 60 页 ［原文 27］

24 注意力机制（Attention） 也是一个数据移动问题 · 第 62 页 ［原文 28］

25 FlashAttention 作为层次结构感知示例 · 第 64 页 ［原文 29］

26 预填充（Prefill） 和解码压力存储器不同 · 第 66 页 ［原文 30］

27个权重、激活（Activations）和KV状态有不同的寿命 · 第 68 页 ［原文 31］

28 训练（Training） 和 推理（Inference） 移动不同的字节 · 第 70 页 ［原文 32］

29 容量和带宽是独立的硬件属性 · 第 72 页 ［原文 33］
30 理论带宽与实际带宽 · 第 74 页 ［原文 34］
31 缓存友好并不自动意味着速度快 · 第 76 页 ［原文 35］
32 高带宽并不意味着低 延迟（Latency） · 第 78 页 ［原文 36］
33 NVIDIA 和 AMD：相同的问题，不同的词汇 · 第 80 页 ［原文 37］
34 代架构改变了细节 · 第 82 页 ［原文 38］
35 种常见误解 · 第 84 页 ［原文 39］
36 实用记忆 理解框架（Mental Model） · 第 86 页 ［原文 40］
参考文献和进一步阅读 · 第 88 页 ［原文 41］
参考图：针对哪个问题阅读什么内容 · 第 91 页 ［原文 42］

# GPU 内存层次结构（GPU Memory Hierarchy）

对应原文第 1 页；点击查看相邻原文对照页

了解 GPU 内存层次结构（GPU Memory Hierarchy） 从 寄存器（Registers） 和 共享内存（Shared Memory） 到 L2、HBM 和 数据移动（Data Movement） 有关 AI 工作负载在何处保存数据以及这些字节如何移动的技术手册：

寄存器（Registers）、共享内存（Shared Memory）、L1/L2 缓存、HBM、局部性、工作集、合并访问（Coalescing）、分块（Tiling） 以及现代 GPU 内核（Kernels） 背后的内存行为。

# Understanding the GPU Memory Hierarchy

From Registers and Shared Memory to L2, HBM, and Data Movement

---

A technical handbook on where AI workloads keep data and how those bytes move: registers, shared memory, L1/L2 caches, HBM, locality, working sets, coalescing, tiling, and the memory behavior behind modern GPU kernels.

@techNmak

# 如何阅读本手册

对应原文第 2 页；点击查看相邻原文对照页

当四个问题分开时，如何阅读本手册 GPU 内存是最容易推理出来的：

1.什么是逻辑内存空间？CUDA命名空间如寄存器（Register）、local、shared和全局内存（Global Memory）。这些名称描述了可见性、生命周期和编程语义，而不是简单的物理阶梯。[1]

2. 数据可以从哪里提供？全局负载可以通过 流式多处理器（SM） 侧 L1/TEX 路径、GPU 范围的 二级缓存（L2 Cache） 或 设备内存（Device Memory） 来满足。 local-内存访问（Memory Access） 在范围内是线程私有的，但仍然使用设备内存支持。[1][3]

3.实际移动了多少字节？张量（Tensor）大小是容量。流量是通过特定接口传输的数量。当重复使用或重复获取数据时，两者可能存在数量级的差异。

4. 是什么限制了进展？容量、延迟（Latency）、带宽、请求 吞吐量（Throughput）、指令发布、同步（Synchronization） 和计算 吞吐量（Throughput） 是不同的约束。 内核（Kernel） 可以舒适地安装在 HBM 中，并且仍然花费大部分时间等待内存移动。

核心（Core） 区别。 内存层次结构（Memory Hierarchy） 不仅仅是从“快”到“慢”的列表。 性能（Performance） 取决于哪些字节移动、移动的频率、哪个级别提供它们以及是否存在足够的独立工作来重叠等待。

范围。本手册建立了理解人工智能系统其余部分所需的内存 理解框架（Mental Model）。 算术强度（Arithmetic Intensity）、Roofline 分析、详细的 合并访问（Coalescing） 规则、共享内存库冲突、寄存器（Register）/占用率（Occupancy） 调整、内核（Kernel） 融合和 性能剖析（Profiling） 稍后都会收到专门的手册。在这里，它们出现的距离仅足以使记忆行为变得连贯。

供应商约定。具体示例使用 NVIDIA CUDA 术语，因为它在 AI 系统中很常见。 AMD CDNA 术语用作交叉检查，因此一般原则不会以 CUDA 法则的形式呈现。[12][16]单位。当计算明确使用 2 的二进制幂时，本手册使用 KiB、MiB 和 GiB。

产品规格使用供应商打印的单位（例如 GB 和 TB/s）复制，而不是默默地转换标签。 [2]

# How to read this handbook

GPU memory is easiest to reason about when four questions are kept separate:

1. **What is the logical memory space?** CUDA names spaces such as register, local, shared, and global memory. Those names describe visibility, lifetime, and programming semantics, not a simple physical ladder.[1]
2. **Where can the data be served from?** A global load may be satisfied by the SM-side L1/TEX path, by the GPU-wide L2 cache, or by device memory. A local-memory access is thread-private in scope but still uses device-memory backing.[1][3]
3. **How many bytes actually move?** Tensor size is capacity. Traffic is the amount transferred across a particular interface. The two can differ by orders of magnitude when data is reused or repeatedly fetched.
4. **What is limiting progress?** Capacity, latency, bandwidth, request throughput, instruction issue, synchronization, and compute throughput are different constraints. A kernel can fit comfortably in HBM and still spend most of its time waiting on memory movement.

**Core distinction.** A memory hierarchy is not merely a list from "fast" to "slow." Performance depends on which bytes move, how often they move, which level supplies them, and whether enough independent work exists to overlap the wait.

**Scope.** This handbook establishes the memory mental model needed by the rest of Understanding AI Systems. Arithmetic intensity, Roofline analysis, detailed coalescing rules, shared-memory bank conflicts, register/occupancy tuning, kernel fusion, and profiling each receive dedicated handbooks later. Here they appear only far enough to make memory behavior coherent.

**Vendor convention.** NVIDIA CUDA terminology is used for concrete examples because it is common in AI systems. AMD CDNA terminology is used as a cross-check so that general principles are not presented as CUDA laws.[12][16]

**Units.** This handbook uses KiB, MiB, and GiB when a calculation explicitly uses binary powers of two. Product specifications are reproduced using the units printed by the vendor, such as GB and TB/s, rather than silently converting the labels.[2]

# 内容

对应原文第 3 页；点击查看相邻原文对照页

目录 1 为什么 GPU 需要 内存层次结构（Memory Hierarchy） 5 2 层次结构一览 6 3 容量、流量、带宽和延迟（Latency） 7 4 寄存器（Registers）：最接近的工作状态 8 5 寄存器（Register） 文件是有限资源 9 6 局部内存（Local Memory） 和 寄存器（Register） 溢出 10 7 共享内存（Shared Memory）：显式片上存储 11 8 共享内存（Shared Memory） 不仅仅是另一个缓存 12 9 L1：一个小型的、依赖于架构的第一缓存 13 10 L2：HBM 之前的器件范围重用 14 11 HBM 和 设备内存（Device Memory） 15 12 HBM 不是 全局内存（Global Memory） 的同义词 16 13 负载在层次结构中的旅程 17 14 时间局部性 18 15 空间局部性 19 16 工作集和重用距离 20 17 缓存命中、未命中和下游流量 21 18 合并访问（Coalescing）：组织 线程束（Warp） 的内存请求 22 19 工作示例：连续和跨步加载 23 20 分块（Tiling） 将容量转化为重用 24 21 通用矩阵乘法（GEMM） 也是一个内存重用问题 25 22 字节移动与算术执行 26 23 为什么按元素 内核（Kernels） 和通用矩阵乘法（GEMM） 的行为不同 27 24 注意力机制（Attention） 也是一个数据移动问题 28

# Contents


| | | |
|---|---|---|
| 1 | Why a GPU needs a memory hierarchy | 5 |
| 2 | The hierarchy at a glance | 6 |
| 3 | Capacity, traffic, bandwidth, and latency | 7 |
| 4 | Registers: the closest working state | 8 |
| 5 | Register files are finite resources | 9 |
| 6 | Local memory and register spills | 10 |
| 7 | Shared memory: explicit on-chip storage | 11 |
| 8 | Shared memory is not just another cache | 12 |
| 9 | L1: a small, architecture-dependent first cache | 13 |
| 10 | L2: device-wide reuse before HBM | 14 |
| 11 | HBM and device memory | 15 |
| 12 | HBM is not a synonym for global memory | 16 |
| 13 | A load's journey through the hierarchy | 17 |
| 14 | Temporal locality | 18 |
| 15 | Spatial locality | 19 |
| 16 | Working sets and reuse distance | 20 |
| 17 | Cache hits, misses, and downstream traffic | 21 |
| 18 | Coalescing: organize a warp's memory requests | 22 |
| 19 | Worked example: contiguous and strided loads | 23 |
| 20 | Tiling turns capacity into reuse | 24 |
| 21 | GEMM is a memory-reuse problem too | 25 |
| 22 | Bytes moved versus arithmetic performed | 26 |
| 23 | Why elementwise kernels and GEMM behave differently | 27 |
| 24 | Attention is also a data-movement problem | 28 |

# 内容续

对应原文第 4 页；点击查看相邻原文对照页

内容续 25 FlashAttention 作为层次结构感知示例 29 26 预填充（Prefill） 和以不同方式解码压力内存 30 27 权重、激活（Activations） 和 KV 状态具有不同的生命周期 31 28 训练（Training） 和 推理（Inference） 移动不同的字节 32 29 容量和带宽是独立的硬件属性33 30 理论带宽与实际带宽 34 31 缓存友好并不意味着快 35 32 高带宽并不意味着低 延迟（Latency） 36 33 NVIDIA 和 AMD：同样的问题，不同的 词表（Vocabulary） 37 34 各代架构改变了细节 38 35 常见误解 39 36 实用内存 理解框架（Mental Model） 40

参考资料和进一步阅读 41 参考图：针对哪个问题阅读什么内容 42 第 1-32 章重点关注持久概念。第 33 章翻译了不同供应商的术语。第 34 章是专门针对 代际（Generation） 的，当硬件文档发生更改时应重新检查。

# Contents continued


| | | |
|---|---|---|
| 25 | FlashAttention as a hierarchy-aware example | 29 |
| 26 | Prefill and decode stress memory differently | 30 |
| 27 | Weights, activations, and KV state have different lifetimes | 31 |
| 28 | Training and inference move different bytes | 32 |
| 29 | Capacity and bandwidth are independent hardware properties | 33 |
| 30 | Theoretical bandwidth versus achieved bandwidth | 34 |
| 31 | Cache-friendly does not automatically mean fast | 35 |
| 32 | High bandwidth does not mean low latency | 36 |
| 33 | NVIDIA and AMD: same problem, different vocabulary | 37 |
| 34 | Architecture generations change the details | 38 |
| 35 | Common misconceptions | 39 |
| 36 | A practical memory mental model | 40 |
| | References and further reading | 41 |
| | Reference map: what to read for which question | 42 |


Chapters 1–32 focus on durable concepts. Chapter 33 translates terminology across vendors. Chapter 34 is deliberately generation-specific and should be rechecked when hardware documentation changes.

# 1 为什么 GPU 需要 内存层次结构（Memory Hierarchy）

对应原文第 5 页；点击查看相邻原文对照页

1 为什么GPU需要内存层次结构（Memory Hierarchy） GPU可以执行大量算术运算，但算术单元只能处理已到达它们的数据。因此，存储系统必须立即解决相互冲突的目标：为大型 模型（Models） 保留足够的总字节，以高聚合速率传送这些字节，并使频繁重用的 值（Values） 靠近执行单元可用。

没有任何一种内存技术能够同时满足这三点。非常大的内存池在物理上距离算术管道较远，并且在与小型片上存储相同的访问特性下提供的成本昂贵。

管道附近的非常小的结构可能非常有用，但它们不能容纳整个 模型（Model） 甚至不能容纳大型 激活（Activation） 张量（Tensor）。

因此，现代 GPU 使用存储层。线程私有的 值（Values） 通常存在于 寄存器（Registers） 中。块中的线程可以通过 共享内存（Shared Memory） 显式协作。硬件管理的缓存无需显式分段即可捕获一些重用。 设备内存（Device Memory），通常是数据中心 AI 加速器上的 HBM，提供更大的容量和非常高的聚合带宽。[1][5][6]重要的词是重用。如果从 设备内存（Device Memory） 获取 值（Value） 并使用一次，则层次结构几乎没有机会摊销该移动。如果相同的 值（Value） 在保留在芯片上时可以使用数十或数百次，则相对昂贵的传输可以提供大量计算。

这就是为什么具有相同数学运算次数的两个 内核（Kernels） 可以以截然不同的速度运行。一个映射可能会不断地从 设备内存（Device Memory） 获取数据。另一种可能会重新组织工作，以便每个获取的图块在被替换之前参与许多操作。

理解框架（Mental Model）。 GPU 内存 性能（Performance） 主要是一个将有用的重用放置在层次结构的正确级别，同时避免级别之间不必要的流量的问题。

本手册的其余部分从最靠近线程的存储向外构建 模型（Model） 到 设备内存（Device Memory）。

# 1 Why a GPU needs a memory hierarchy

A GPU can execute an enormous number of arithmetic operations, but arithmetic units can only work on data that has reached them. The storage system therefore has to solve conflicting goals at once: keep enough total bytes for large models, deliver those bytes at high aggregate rates, and make frequently reused values available close to the execution units.

No single memory technology does all three well. Very large memory pools are physically farther from the arithmetic pipelines and are expensive to provision at the same access characteristics as tiny on-chip storage. Very small structures near the pipelines can be extremely useful, but they cannot hold an entire model or even a large activation tensor.

Modern GPUs therefore use layers of storage. Thread-private values commonly live in registers. Threads in a block can explicitly cooperate through shared memory. Hardware-managed caches capture some reuse without explicit staging. Device memory, often HBM on datacenter AI accelerators, provides much larger capacity and very high aggregate bandwidth.[1][5][6]

The important word is *reuse*. If a value is fetched from device memory and used once, the hierarchy has little opportunity to amortize that movement. If the same value can be used dozens or hundreds of times while it remains on chip, a relatively expensive transfer can feed a large amount of computation.

This is why two kernels with the same mathematical operation count can run at very different speeds. One mapping may continually fetch data from device memory. Another may reorganize the work so that each fetched tile participates in many operations before it is replaced.

**Mental model.** GPU memory performance is largely a question of placing useful reuse at the right level of the hierarchy while avoiding unnecessary traffic between levels.

The rest of this handbook builds that model from the storage closest to a thread outward to device memory.

# 2 层次结构一览

对应原文第 6 页；点击查看相邻原文对照页

2 层次结构一览 下图是有意示意性的。它展示了人工智能数据的通用路径，而不是保证每个 GPU 上的每条指令都完全遵循这些框。

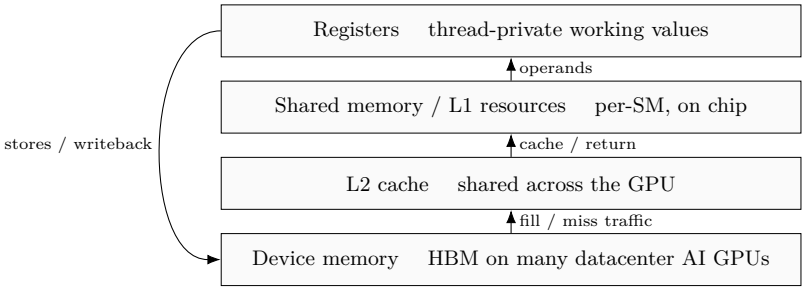
寄存器（Registers） 线程私有工作 值（Values） 共享内存（Shared Memory） / L1 资源每个 流式多处理器（SM），片上 二级缓存（L2 Cache） 在 GPU 上共享 设备内存（Device Memory） 许多数据中心 AI GPU 上的 HBM 填充/错过流量缓存/返回操作数存储/写回 三个注意事项使此图保持真实。

首先，编程空间和物理结构并不相同。 CUDA 全局内存（Global Memory） 是整个网格可见的地址空间。全局负载可以命中片上缓存。 CUDA 局部内存（Local Memory） 是一个线程私有的，但其后备存储位于设备内存（Device Memory） 中。[1][3]其次，层次结构依赖于体系结构。在最新的 NVIDIA 数据中心 GPU 上，L1/纹理缓存（Texture Cache） 和 共享内存（Shared Memory） 来自统一的片上资源，其分区可以调整。 AMD CDNA 公开了具有每 CU L1 缓存和本地数据共享 (LDS) 的不同组织。[5][6][12]第三，更接近并不意味着自动更好。将数据移入 共享内存（Shared Memory） 需要指令，并且通常需要 同步（Synchronization）。在寄存器（Registers）中保留更多的值（Values）可以增加寄存器压力（Register Pressure）。缓存可能包含从未重用的数据。仅当访问模式值得其成本时，每个级别才有帮助。

将层次结构视为一组可以保留和重用数据的位置，而不是每个级别有一个固定 延迟（Latency） 编号的通用排名。

# 2 The hierarchy at a glance

The diagram below is intentionally schematic. It shows a common path for AI data, not a promise that every instruction on every GPU follows exactly these boxes.




Three cautions keep this picture honest.

First, **programming spaces and physical structures are not identical**. CUDA global memory is an address space visible across the grid. A global load can hit an on-chip cache. CUDA local memory is private to one thread, yet its backing storage is in device memory.[1][3]

Second, **the hierarchy is architecture-dependent**. On recent NVIDIA datacenter GPUs, the L1/texture cache and shared memory draw from a unified on-chip resource whose partition can be adjusted. AMD CDNA exposes a different organization with per-CU L1 caches and Local Data Share (LDS).[5][6][12]

Third, **closer does not mean automatically better**. Moving data into shared memory costs instructions and often synchronization. Keeping more values in registers can increase register pressure. A cache may contain data that is never reused. Each level helps only when the access pattern makes its cost worthwhile.

> Think of the hierarchy as a set of places where data can be retained and reused, not as a universal ranking with one fixed latency number per level.

# 3 容量、流量、带宽和 延迟（Latency）

对应原文第 7 页；点击查看相邻原文对照页

3 容量、流量、带宽和 延迟（Latency） 这四个量通常被归结为模糊的词“内存”。他们回答不同的问题。

容量是一个存储级别可以容纳多少字节。 BF16 中形状为 [4, 4096, 8192] 的 张量（Tensor） 包含

这是存储要求，而不是关于 内核（Kernel） 运行时将移动多少数据的声明。

流量是通过特定接口传输的字节数。如果读取不由缓存或其他重用机制提供服务，则 256 MiB 张量（Tensor） 读取十次可以创建远远超过 256 MiB 的下游流量。

带宽是一个速率：

带宽 = 字节传输时间。

峰值带宽是特定条件下的硬件上限。获得的带宽取决于访问模式、请求并发性、缓存行为、争用以及 内核（Kernel） 本身。[2][3] 延迟（Latency） 是启动访问和将其结果提供给相关工作之间的延迟。系统可以具有高带宽，而个人访问仍然有大量的 延迟（Latency）。 GPU 通过保持许多独立操作的运行并在某些请求等待时安排其他工作来部分容忍这种情况。

数量 所回答的问题 容量 这里可以驻留多少数据？

流量 通过此接口有多少字节？

带宽 字节随时间变化的速度有多快？

延迟（Latency） 需要多长时间才能满足特定的依赖性？

工作负载可以适合 HBM，但仍然受到带宽限制。它可以实现高带宽但仍然受到影响

永远不应该用作同义词。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 3 Capacity, traffic, bandwidth, and latency

These four quantities are often collapsed into the vague word “memory.” They answer different questions.

**Capacity** is how many bytes a storage level can hold. A tensor with shape $[4, 4096, 8192]$ in BF16 contains

$$4 \times 4096 \times 8192 \times 2 = 268{,}435{,}456 \text{ bytes} = 256 \text{ MiB}.$$

That is a storage requirement, not a statement about how much data will move while a kernel runs.

**Traffic** is the number of bytes transferred across a particular interface. A 256 MiB tensor read ten times can create far more than 256 MiB of downstream traffic if the reads are not served by a cache or other reuse mechanism.

**Bandwidth** is a rate:

$$\text{bandwidth} = \frac{\text{bytes transferred}}{\text{time}}.$$

Peak bandwidth is a hardware ceiling under specified conditions. Achieved bandwidth depends on the access pattern, request concurrency, cache behavior, contention, and the kernel itself.[2][3]

**Latency** is the delay between initiating an access and having its result available to the dependent work. A system can have high bandwidth while an individual access still has substantial latency. GPUs tolerate this partly by keeping many independent operations in flight and scheduling other work while some requests wait.

| Quantity | Question it answers |
|---|---|
| Capacity | How much data can reside here? |
| Traffic | How many bytes crossed this interface? |
| Bandwidth | How quickly can bytes cross over time? |
| Latency | How long until a particular dependency can be satisfied? |

A workload can fit in HBM and still be bandwidth-limited. It can achieve high bandwidth and still suffer from latency when it exposes too little independent work. Capacity, traffic, bandwidth, and latency should never be used as synonyms.

# 4 寄存器（Registers）：最接近的工作状态

对应原文第 8 页；点击查看相邻原文对照页

4 寄存器（Registers）：最接近的工作状态 寄存器（Registers） 保存线程直接执行指令所需的 值（Values）：循环计数器、指针、矩阵（Matrix） 块的片段、中间算术结果和累加器。 CUDA 自动标量变量通常驻留在 寄存器（Registers） 中，除非编译器选择其他位置。[1]

寄存器（Registers） 不是线程共同浏览的可寻址缓存。它们属于各个线程的执行状态。一个线程不能通过普通内存地址简单地加载另一个线程的 寄存器（Register） 值（Value）。因此，线程之间的通信使用其他机制，例如 共享内存（Shared Memory）、线程束（Warp） 级操作或特定于体系结构的集体指令。

这使得 寄存器（Registers） 成为非常细粒度的重用的理想选择。考虑一个线程累积部分 点积（Dot Product）：

a←a +西依。

将 a 保留在 寄存器（Register） 中意味着每个乘法累加可以更新相同的附近状态，而无需在每次迭代后将部分和写入 设备内存（Device Memory）。

矩阵（Matrix） 内核（Kernels） 进一步推动了这一点。 CUTLASS 描述了 线程束（Warp） 和线程级 通用矩阵乘法（GEMM） 结构，其中操作数从 共享内存（Shared Memory） 加载到 寄存器（Register） 支持的片段中，并且累加器在许多 矩阵（Matrix） 指令中保留在 寄存器（Registers） 中。 [14]

好处并不是“寄存器（Registers） 是免费的”。它们是有限的硬件资源，并且参与调度约束。但是，当 值（Value） 在一个线程内具有较高的重用性时，将其保持在 寄存器（Register） 状态可以避免通过较大内存结构的重复流量。

安全措辞。 寄存器（Registers） 是接近执行的线程私有工作 值（Values） 的正常位置。

避免通用声明，例如“寄存器（Register） 访问始终是一个周期”。确切的时序取决于指令路径、依赖性、存储、调度和架构。

本手册将 寄存器（Registers） 视为内存层次结构级别，因为它们确定临时状态是保持接近计算还是溢出到可寻址内存系统。 占用率（Occupancy） 结果在 AS09 中得到专门处理。

# 4 Registers: the closest working state

Registers hold the values a thread needs directly for instruction execution: loop counters, pointers, fragments of matrix tiles, intermediate arithmetic results, and accumulators. CUDA automatic scalar variables generally reside in registers unless the compiler chooses another placement.[1]

Registers are not an addressable cache that threads browse collectively. They belong to the execution state of individual threads. One thread cannot simply load another thread's register value through an ordinary memory address. Communication between threads therefore uses other mechanisms such as shared memory, warp-level operations, or architecture-specific collective instructions.

This makes registers ideal for very fine-grained reuse. Consider a thread accumulating a partial dot product:

$$a \leftarrow a + x_i y_i.$$

Keeping $a$ in a register means each multiply-accumulate can update the same nearby state without writing the partial sum to device memory after every iteration.

Matrix kernels push this much further. CUTLASS describes warp- and thread-level GEMM structures in which operands are loaded from shared memory into register-backed fragments and accumulators remain in registers across many matrix instructions.[14]

The benefit is not that "registers are free." They are a finite hardware resource and they participate in scheduling constraints. But when a value has high reuse inside one thread, keeping it in register state avoids repeated traffic through larger memory structures.

**Safe wording.** Registers are the normal home for thread-private working values close to execution. Avoid universal claims such as "a register access is always one cycle." Exact timing depends on the instruction path, dependencies, banking, scheduling, and architecture.

This handbook treats registers as a memory-hierarchy level because they determine whether temporary state remains close to compute or spills into the addressable memory system. The occupancy consequences receive a dedicated treatment in AS09.

# 5 寄存器（Register） 文件是有限资源

对应原文第 9 页；点击查看相邻原文对照页

5 寄存器（Register） 文件是有限资源 寄存器文件（Register File） 的总量可能很大，但仍然成为一个约束，因为数千个 驻留（Resident） 线程共享它。 NVIDIA 当前的计算能力表列出了几种最新架构的每个 流式多处理器（SM） 64K 32 位 寄存器（Registers），而 Hopper H100 每个线程最多允许 255 个 寄存器（Registers）。[4][5]假设 内核（Kernel） 每个线程使用 128 个 寄存器（Registers） 并启动 256 个线程块。暂时忽略分配粒度和其他架构限制，一个块将请求

32 位 寄存器（Registers）。 64K-寄存器（Register） 流式多处理器（SM） 无法同时托管任意数量的此类块。因此，即使共享内存使用量很小，寄存器（Register） 的需求也会影响 驻留（Resident） 的工作量。

这造成了真正的权衡。每个线程更多的 寄存器（Registers） 可以使有用的临时变量和累加器靠近算术管道。但较大的 寄存器（Register） 占用空间可以减少 驻留（Resident） 线程束（Warps） 的数量，从而可以减少可用于隐藏摊位的独立工作池。

相反的策略也可能适得其反。强制编译器使用较少的 寄存器（Registers） 可能会增加 占用率（Occupancy），但会创建额外的本地 内存流量（Memory Traffic），因为 值（Values） 不再适合 寄存器（Register） 状态。在这种情况下，内核（Kernel） 可能会获得更多的 驻留（Resident） 线程束（Warps），但每个 线程束（Warp） 会执行更多的内存工作。

寄存器压力（Register Pressure） 本质上并不是坏事。这是一种资源权衡。相关问题是，寄存器（Register） 占用空间是否能够实现有用的重用和指令级并行性，而不会产生足够的 溢出（Spilling） 或 驻留状态（Residency） 损失来损害 内核（Kernel）。

这就是为什么“使用更少的 寄存器（Registers）”和“最大化 占用率（Occupancy）”不是通用优化规则。AS09 将直接处理该交互。这里的记忆教训更简单：有限的 寄存器（Register） 预算决定了有多少线程私有状态可以保留在层次结构的最接近的级别。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 5 Register files are finite resources

A register file can be large in aggregate and still become a constraint because thousands of resident threads share it. NVIDIA's current compute-capability tables list 64K 32-bit registers per SM for several recent architectures, while Hopper H100 allows at most 255 registers per thread.[4][5]

Suppose a kernel uses 128 registers per thread and launches 256-thread blocks. Ignoring allocation granularity and other architectural limits for the moment, one block would request

$$128 \times 256 = 32{,}768$$

32-bit registers. A 64K-register SM cannot host an arbitrary number of such blocks simultaneously. Register demand therefore affects how much work can be resident even when shared-memory use is tiny.

This creates a real trade-off. More registers per thread can keep useful temporaries and accumulators close to the arithmetic pipelines. But a larger register footprint can reduce the number of resident warps, which can reduce the pool of independent work available to hide stalls.

The opposite strategy can also backfire. Forcing a compiler to use fewer registers may increase occupancy, yet create extra local-memory traffic because values no longer fit in register state. In that case a kernel may gain more resident warps but do more memory work per warp.

> Register pressure is not inherently bad. It is a resource trade-off. The relevant question is whether the register footprint enables useful reuse and instruction-level parallelism without creating enough spilling or residency loss to hurt the kernel.

This is why "use fewer registers" and "maximize occupancy" are not general optimization rules. AS09 will treat that interaction directly. Here the memory lesson is simpler: a finite register budget decides how much thread-private state can remain at the closest level of the hierarchy.

# 6 局部内存（Local Memory） 和 寄存器（Register） 泄漏

对应原文第 10 页；点击查看相邻原文对照页

6 局部内存（Local Memory） 和 寄存器（Register） 溢出 CUDA 的术语 局部内存（Local Memory） 很容易被误读。“本地”描述了范围：每个线程都有自己的本地地址空间。这并不意味着存储在物理上靠近该线程的执行单元。当前 CUDA 文档将 局部内存（Local Memory） 置于 设备内存（Device Memory） 中。[1]

编译器可以将 局部内存（Local Memory） 用于具有动态索引的数组、大型线程私有对象或不能保留在 寄存器（Registers） 中的 值（Values）。最后一种情况通常称为寄存器 溢出（Spilling）。[1][3]从概念上讲：

线程临时数据不会保留在 寄存器（Registers） ----------→本地地址空间中。

当数据存在时，本地 内存访问（Memory Access） 可以由缓存提供服务。然而，如果未命中，请求将在层次结构中进一步传播至 设备内存（Device Memory）。这就是为什么溢出并不等同于“每次一次 HBM 访问”，但它仍然会增加负载、存储、缓存压力和长 延迟（Latency） 依赖项，而这些依赖项在 值（Value） 停留在 寄存器（Register） 中时不存在。

线程私有范围并不意味着分散的物理请求。 CUDA 跨通道布置本地存储，以便其线程访问相同的每线程偏移量的 线程束（Warp） 仍然可以生成 合并访问（Coalesced Access）。例如，即使每个通道拥有不同的逻辑对象，每个通道读取其自己的溢出数组的相同槽都可以得到有效的服务。[1]

寄存器（Register） 值（Value） 本地地址空间 L1/L2 缓存 设备内存（Device Memory） 溢出 不要仅仅因为它是线程本地的，就在物理层次结构中的 寄存器（Registers） 旁边绘制“局部内存（Local Memory）”。

它的逻辑范围是本地的。它的后备存储位于设备内存系统中。

# 6 Local memory and register spills

CUDA's term *local memory* is easy to misread. "Local" describes scope: each thread has its own local address space. It does not mean the storage is physically next to that thread's execution units. Current CUDA documentation places local memory in device memory.[1]

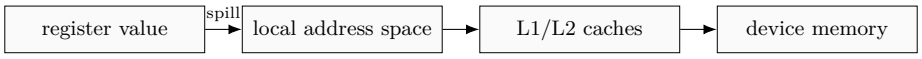
The compiler may use local memory for arrays with dynamic indexing, large thread-private objects, or values that cannot remain in registers. The last case is usually called **register spilling**.[1][3]

Conceptually:

$$\text{thread temporary} \xrightarrow{\text{does not remain in registers}} \text{local address space.}$$

A local-memory access can be served by cache when the data is present there. On a miss, however, the request travels farther through the hierarchy toward device memory. That is why a spill is not equivalent to "one HBM access every time," but it can still add loads, stores, cache pressure, and long-latency dependencies that did not exist when the value stayed in a register.

Thread-private scope does not imply a scattered physical request. CUDA lays out local storage across lanes so that a warp whose threads access the same per-thread offset can still generate a coalesced access. For example, every lane reading the same slot of its own spilled array can be serviced efficiently even though each lane owns a distinct logical object.[1]




Do not draw "local memory" next to registers in a physical hierarchy merely because it is thread-local. Its logical scope is local. Its backing storage is in the device-memory system.

# 7 共享内存（Shared Memory）：显式片上存储

对应原文第 11 页；点击查看相邻原文对照页

7 共享内存（Shared Memory）：显式片上存储 共享内存（Shared Memory） 为块中的线程提供显式管理的位置来交换和重用数据。 CUDA 通过块作用域公开它，并将存储放置在 流式多处理器（SM） 上。在最新的 NVIDIA GPU 上，它是从 L1/纹理缓存使用的相同统一片上资源中提取的。[1][5][6]这种显式控制就是定义属性。硬件缓存自动决定保留哪些行。

共享内存（Shared Memory） 根据 内核（Kernel） 的程序逻辑进行填充、索引、同步和覆盖。

一个常见的模式是：

1.从设备内存（Device Memory）协同加载一个图块，

2. 同步以便图块准备就绪，

3. 在许多操作中重复使用图块，

4.工作完成后，将其更换为下一块瓷砖。

回报来自重用。如果 256 个线程每个都需要 值（Values），而许多相邻线程也需要该 值（Values），则获取一个共享图块可以替代大量冗余的下游负载。

共享内存（Shared Memory） 也有成本。它消耗有限的每 流式多处理器（SM） 容量，可以减少块 驻留状态（Residency），在线程通信时需要正确的 同步（Synchronization），并且具有可以序列化某些模式的存储访问行为。这些银行冲突详细信息属于 AS08。

Hopper H100 和 Blackwell B200 均支持 256 KB 统一 L1/纹理/共享资源内的可配置共享内存分割，每个 流式多处理器（SM） 具有高达 228 KB 的共享内存容量，并且由这些特定设备上的一个选择加入的块可寻址高达 227 KB。[5][6]一般原理，具体实现。“共享内存（Shared Memory）是程序员管理的片上存储”是持久的。确切的容量、剥离选择、银行行为和集群级功能取决于 GPU 代际（GPU Generation）。

# 7 Shared memory: explicit on-chip storage

Shared memory gives threads in a block an explicitly managed place to exchange and reuse data. CUDA exposes it with block scope and places the storage on the SM. On recent NVIDIA GPUs it is carved from the same unified on-chip resource used by L1/texture caching.[1][5][6]

That explicit control is the defining property. A hardware cache decides automatically which lines to retain. Shared memory is filled, indexed, synchronized, and overwritten according to the kernel's program logic.

A common pattern is:

1. cooperatively load a tile from device memory,
2. synchronize so the tile is ready,
3. reuse the tile across many operations,
4. replace it with the next tile when the work is finished.

The payoff comes from reuse. If 256 threads each need values that many neighboring threads also need, fetching one shared tile can replace a large number of redundant downstream loads.

Shared memory also has costs. It consumes finite per-SM capacity, can reduce block residency, requires correct synchronization when threads communicate, and has banked access behavior that can serialize some patterns. Those bank-conflict details belong to AS08.

Hopper H100 and Blackwell B200 both support configurable shared-memory carveouts within a 256 KB unified L1/texture/shared resource, with up to 228 KB of shared-memory capacity per SM and up to 227 KB addressable by one opted-in block on those specific devices.[5][6]

**General principle, specific implementation.** "Shared memory is programmer-managed on-chip storage" is durable. Exact capacity, carveout choices, bank behavior, and cluster-level features depend on GPU generation.

# 8 共享内存（Shared Memory） 不仅仅是另一个缓存

对应原文第 12 页；点击查看相邻原文对照页

8 共享内存（Shared Memory） 不仅仅是另一个缓存 共享内存（Shared Memory） 和缓存都可以使数据靠近计算，但它们解决问题的方式不同。

共享内存（Shared Memory） 硬件缓存 放置 内核（Kernel） 选择暂存内容 硬件对访问做出反应 寻址 显式共享内存地址 普通地址空间 生命周期 由 内核（Kernel）/块逻辑管理 由替换策略管理 通信 可以在协作线程之间交换数据 主要服务于内存访问 主要风险 同步（Synchronization）、存储体冲突、容量压力 未命中、驱逐、有限位置 调用共享内存（Shared Memory）“缓存”有时可以用作类比，因为程序员经常使用它来保留否则会重复获取的数据。但如果这种类比隐藏了软件责任，就会产生误导。

例如，矩阵（Matrix） 内核（Kernel） 可以准确地知道 线程块（Thread Block） 将使用哪一个 A 和 B 块。将这些图块暂存在 共享内存（Shared Memory） 中可实现确定性重用。通用硬件缓存不接收该高级意图。它看到内存请求流并应用自己的策略。

另一方面，明确的分期并不总是值得做的。如果数据几乎没有重用，如果访问已经在缓存中命中，或者如果暂存添加了太多 同步（Synchronization） 和复制，则共享内存版本可能会比更简单的缓存路径慢。

正确的区别是控制。 共享内存（Shared Memory） 公开了软件控制的重用机制。缓存公开了硬件控制的重用机制。它们可能在某些架构上共享物理资源，但它们的编程语义仍然不同。

这种区别在 FlashAttention、通用矩阵乘法（GEMM）、转置、归约和许多 融合内核（Fused Kernels） 中变得很重要。 内核（Kernel） 编写器正在决定哪些中间体值得可预测的片上驻留，而不是希望缓存替换策略将它们保留下来。

# 8 Shared memory is not just another cache

Shared memory and cache can both keep data close to compute, but they solve the problem differently.

| | Shared memory | Hardware cache |
|---|---|---|
| Placement | Kernel chooses what to stage | Hardware reacts to accesses |
| Addressing | Explicit shared-memory address | Ordinary address space |
| Lifetime | Managed by kernel/block logic | Managed by replacement policy |
| Communication | Can exchange data among cooperating threads | Primarily serves memory accesses |
| Main risk | Synchronization, bank conflicts, capacity pressure | Misses, eviction, limited locality |

Calling shared memory “a cache” is sometimes useful as an analogy because programmers often use it to retain data that would otherwise be fetched repeatedly. But the analogy becomes misleading if it hides the software responsibility.

For example, a matrix kernel may know exactly which $A$ and $B$ tiles will be used by a thread block. Staging those tiles in shared memory creates deterministic reuse. A general hardware cache does not receive that high-level intent. It sees a stream of memory requests and applies its own policies.

On the other hand, explicit staging is not always worth doing. If data has little reuse, if accesses already hit well in cache, or if the staging adds too much synchronization and copying, a shared-memory version can be slower than a simpler cached path.

> The right distinction is control. Shared memory exposes a software-controlled reuse mechanism. Cache exposes a hardware-controlled reuse mechanism. They may share physical resources on some architectures, but their programming semantics remain different.

This distinction becomes important in FlashAttention, GEMM, transposes, reductions, and many fused kernels. The kernel writer is deciding which intermediates deserve predictable on-chip residence rather than hoping a cache replacement policy keeps them around.

# 9 L1：小型、依赖于架构的第一级缓存

对应原文第 13 页；点击查看相邻原文对照页

9 L1：小型、依赖于体系结构的第一高速缓存 L1 是 流式多处理器（SM） 附近的许多普通内存路径遇到的第一个硬件管理的高速缓存。它的确切组织随着体系结构的不同而变化，不同的内存空间可以以不同的方式与之交互。因此，Nsight Compute 将 L1/TEX 作为一个架构单元，而不是假装存在一种通用的教科书缓存设计。 [3]

在最新的 NVIDIA 数据中心架构中，L1/纹理缓存（Texture Cache） 和共享内存容量是统一片上资源的一部分。 Hopper H100 和 Blackwell B200 具有相同的 256 KB 最大组合容量，而为 共享内存（Shared Memory） 保留的数量可以在支持的剥离选项内根据 内核（Kernel） 进行更改。[5][6]有用的 L1 命中可以减少流向 L2 和 设备内存（Device Memory） 的流量。这很重要，因为下游接口由更多请求者共享，并且距离发出 流式多处理器（SM） 更远。 Nsight 的内存分析准确地报告请求的扇区、L2 未命中、命中率和链接利用率，因为高级“负载计数”并不能揭示有多少流量逃逸第一个缓存。 [3]

但L1命中率并不是通用的性能（Performance）分数。 内核（Kernel） 可以具有很高的命中率，但仍然受到指令问题、序列化、共享内存冲突或某些其他资源的限制。相反，流式 内核（Kernel） 可能有意具有很少的时间重用，并且如果其访问被合并并且设备带宽得到有效利用，则仍然表现良好。

架构警告。不要将一种 L1 大小或一种缓存规则记住为“GPU L1”。持久的想法是，小型片上硬件管理结构可以在消耗 L2 或 HBM 流量之前满足某些请求。必须检查目标架构的确切容量和行为。

# 9 L1: a small, architecture-dependent first cache

L1 is the first hardware-managed cache encountered by many ordinary memory paths near an SM. Its exact organization changes across architectures, and different memory spaces can interact with it differently. Nsight Compute therefore presents L1/TEX as an architectural unit rather than pretending there is one universal textbook cache design.[3]

On recent NVIDIA datacenter architectures, the L1/texture cache and shared-memory capacity are part of a unified on-chip resource. Hopper H100 and Blackwell B200 expose the same 256 KB maximum combined capacity, while the amount reserved for shared memory can be changed per kernel within supported carveout choices.[5][6]

A useful L1 hit can reduce traffic to L2 and device memory. That matters because downstream interfaces are shared by more requesters and are farther from the issuing SM. Nsight's memory analysis reports requested sectors, misses to L2, hit rates, and link utilization precisely because a high-level "load count" does not reveal how much traffic escaped the first cache.[3]

But L1 hit rate is not a universal performance score. A kernel can have a high hit rate and still be limited by instruction issue, serialization, shared-memory conflicts, or some other resource. Conversely, a streaming kernel may intentionally have little temporal reuse and still perform well if its accesses are coalesced and device bandwidth is used efficiently.

**Architecture warning.** Do not memorize one L1 size or one caching rule as "the GPU L1." The durable idea is that a small on-chip hardware-managed structure can satisfy some requests before they consume L2 or HBM traffic. Exact capacity and behavior must be checked for the target architecture.

# 10 L2：HBM 之前的设备范围内重用

对应原文第 14 页；点击查看相邻原文对照页

10 L2：HBM L2 位于层次结构更深处之前的设备范围重用，并在整个 GPU 之间共享，而不是属于一个 流式多处理器（SM）。 Nsight 将其描述为片上内存客户端和 设备内存（Device Memory） 之间的缓存，并报告 L2 请求、扇区、命中率、吞吐量（Throughput） 以及继续发送到设备、系统或对等内存的未命中情况。 [3]

此位置使得 L2 对于跨越块或 流式多处理器（SM） 边界的重用非常有用。当另一个块需要相同区域时，从 流式多处理器（SM） 的 L1 中逐出的图块可能仍可在 L2 中使用。因此，即使软件没有明确协调重用，重复的 模型（Model） 权重读取、共享查找表和重叠的 张量（Tensor） 图块也可以受益。

L2 仍然是有限的。如果 工作集（Working Set） 比缓存大得多，或者如果许多不相关的流竞争它，则有用的行可能会在重用之前被逐出。例如，64 MiB 的热区域不能完全驻留在 H100 标称的 50 MB L2 内。这并不意味着缓存命中为零。它只是意味着整个区域不能同时保留。[5]

从 Ampere 级 CUDA 设备开始，软件还可以为 L2 中选定的全局内存区域提供持久性提示。这些机制影响替换优先级，而不是将 L2 转换为显式寻址的暂存存储器。[1]

L2 可以减少 HBM 流量，而无需更改逻辑 张量（Tensor） 大小。这就是为什么“张量（Tensor） 是 4 GB”几乎没有告诉您在 内核（Kernel） 期间实际穿过 HBM 接口的字节数。

下一个级别是 设备内存（Device Memory） 本身，其中大容量和巨大的聚合带宽共存，但移动成本比片上重用高得多。

# 10 L2: device-wide reuse before HBM

L2 sits deeper in the hierarchy and is shared across the GPU rather than belonging to one SM. Nsight describes it as the cache between on-chip memory clients and device memory, and reports L2 requests, sectors, hit rate, throughput, and misses that continue to device, system, or peer memory.[3]

This position makes L2 useful for reuse that crosses block or SM boundaries. A tile evicted from one SM's L1 may still be available in L2 when another block needs the same region. Repeated model-weight reads, shared lookup tables, and overlapping tensor tiles can therefore benefit even when software does not explicitly coordinate the reuse.

L2 is still finite. If a working set is much larger than the cache, or if many unrelated streams compete for it, useful lines may be evicted before they are reused. A 64 MiB hot region, for example, cannot reside in its entirety inside H100's nominal 50 MB L2. That does not imply zero cache hits. It simply means the entire region cannot be retained simultaneously.[5]

Starting with Ampere-class CUDA devices, software can also provide persistence hints for selected global-memory regions in L2. These mechanisms influence replacement priority rather than converting L2 into explicitly addressed scratchpad memory.[1]

> L2 can reduce HBM traffic without changing the logical tensor size. This is why "the tensor is 4 GB" tells you almost nothing about how many bytes will actually cross the HBM interface during a kernel.

The next level is device memory itself, where large capacity and enormous aggregate bandwidth coexist with much higher movement cost than on-chip reuse.

# 11 HBM 和 设备内存（Device Memory）

对应原文第 15 页；点击查看相邻原文对照页

11 HBM 和设备内存（Device Memory） 高带宽内存（HBM）是许多数据中心加速器使用的物理内存技术。

它旨在提供非常宽的接口和接近 GPU 封装的高聚合带宽。例如，H100 SXM 指定为 80 GB 的 HBM3 和 3.35 TB/s 的 内存带宽（Memory Bandwidth）。 H200 SXM 将其提升至 141 GB HBM3e，速度为 4.8 TB/s。[7][8]与普通 CPU 内存通道相比，这些数字非常巨大，但从 流式多处理器（SM） 的角度来看，HBM 仍然是设备上层次结构的远端。即使算术单元能够执行更多工作，在 HBM 和计算之间重复移动相同 值（Values） 的 内核（Kernel） 也可能会受到该流量的限制。

HBM 容量和HBM 带宽是独立的规格。更多容量可让更大的 模型（Model） 或 工作集（Working Set） 保留在设备上。更大的带宽可以让内存系统维持更高的传输速率。

增加一个在数学上并不意味着另一个。

HBM 也不是通用的。一些 GPU 使用 GDDR 而不是 HBM。保持稳定的编程概念是设备内存：支持全局分配和其他地址空间的大型片外内存。

HBM 是当前许多人工智能加速器中这一角色的实现之一。

HBM 中的 模型（Model） 配件回答了容量问题。它没有回答权重、激活（Activations） 或 KV 状态将被读取多少次，访问是否会命中缓存，或者是否可以在 延迟（Latency） 目标内维持所需的流量。

这种区别在 推理（Inference） 中尤其重要，其中 模型（Model） 可以轻松地放入内存中，但在解码期间仍然受到重复流权重或 KV 状态的限制。

# 11 HBM and device memory

High Bandwidth Memory, or HBM, is a physical memory technology used by many datacenter accelerators. It is designed to provide very wide interfaces and high aggregate bandwidth close to the GPU package. H100 SXM, for example, is specified with 80 GB of HBM3 and 3.35 TB/s of memory bandwidth. H200 SXM raises this to 141 GB of HBM3e at 4.8 TB/s.[7][8]

Those numbers are enormous compared with ordinary CPU memory channels, but HBM is still the far end of the on-device hierarchy from an SM's perspective. A kernel that repeatedly moves the same values between HBM and compute can become limited by that traffic even when the arithmetic units are capable of much more work.

HBM capacity and HBM bandwidth are independent specifications. More capacity lets a larger model or working set remain on the device. More bandwidth lets the memory system sustain a higher rate of transfers. Increasing one does not mathematically imply the other.

HBM is also not universal. Some GPUs use GDDR rather than HBM. The programming concept to keep stable is **device memory**: the large off-chip memory backing global allocations and other address spaces. HBM is one implementation of that role in many current AI accelerators.

> A model fitting in HBM answers a capacity question. It does not answer how many times the weights, activations, or KV state will be read, whether the accesses will hit cache, or whether the required traffic can be sustained within the latency target.

This distinction is especially important in inference, where a model may fit comfortably in memory yet still be limited by repeatedly streaming weights or KV state during decode.

# 12 HBM 不是 全局内存（Global Memory） 的同义词

对应原文第 16 页；点击查看相邻原文对照页

12 HBM 不是 全局内存（Global Memory） 的同义词 CUDA 的 全局内存（Global Memory） 是跨线程可见的编程 模型（Model） 地址空间。 HBM 是一种物理内存技术。将它们混为一谈会产生几个错误的结论。

来自 全局内存（Global Memory） 的负载不一定到达 HBM。如果所请求的数据已经存在于合适的高速缓存中，则可以在层次结构中较早地满足该请求。相反，来自线程私有本地地址空间的数据最终可以由用于全局分配的相同 设备内存（Device Memory） 支持。[1][3]其区别可写为：

地址空间语义==此特定访问的物理服务位置。

这在解释分析器时很重要。 内核（Kernel） 可以执行许多全局加载指令，同时生成更少的 HBM 流量，因为 L1 或 L2 提供重复的请求。相反的情况也可能发生：合并不佳或重用率低的访问可能会产生比源代码隐含的有用字节多得多的下游流量。

在讨论“内存使用”时它也很重要。 模型（Model） 分配的全局 内存占用（Memory Footprint） 是一个容量度量。缓存 占用率（Occupancy） 是暂时的硬件状态。 寄存器（Register） 和共享内存的使用是每个 内核（Kernel） 的资源要求。它们不应该像一个平池一样随意地加在一起。

实用语言。在讨论 CUDA 地址空间时说“全局内存加载”。在讨论字节的物理服务或传输位置时，请说“L1 hit”、“L2 hit”或“HBM/device-内存流量（Memory Traffic）”。

本手册的其余部分一致地使用了这种区别。它让我们讨论单个 张量（Tensor），同时单独推理其逻辑位置、缓存行为和通过硬件的实际移动。

# 12 HBM is not a synonym for global memory

CUDA's *global memory* is a programming-model address space visible across threads. HBM is a physical memory technology. Conflating them produces several wrong conclusions.

A load from global memory does not necessarily reach HBM. If the requested data is already present in a suitable cache, the request can be satisfied earlier in the hierarchy. Conversely, data from the thread-private local address space can ultimately be backed by the same device memory used for global allocations.[1][3]

The distinction can be written as:

$$\text{address-space semantics} \neq \text{where this particular access is physically served.}$$

This matters when interpreting profilers. A kernel can execute many global-load instructions while generating much less HBM traffic because L1 or L2 supplies repeated requests. The reverse can also happen: poorly coalesced or low-reuse accesses may produce far more downstream traffic than the useful bytes implied by the source code.

It also matters when discussing "memory usage." A model's allocated global-memory footprint is a capacity measure. Cache occupancy is a transient hardware state. Register and shared-memory use are per-kernel resource requirements. They should not be added together casually as if they were one flat pool.

**Practical language.** Say "global-memory load" when discussing the CUDA address space. Say "L1 hit," "L2 hit," or "HBM/device-memory traffic" when discussing where bytes were physically served or transferred.

The rest of this handbook uses that distinction consistently. It lets us talk about a single tensor while separately reasoning about its logical location, cache behavior, and actual movement through hardware.

# 13 负载在层次结构中的旅程

对应原文第 17 页；点击查看相邻原文对照页

13 负载在层次结构中的旅程 对于最新 NVIDIA GPU 上的普通可缓存全局负载，一个有用的教学路径是：

线程束（Warp） 问题加载 L1/TEX L2 设备内存（Device Memory） miss miss hit returns data hit returns data 这是 理解框架（Mental Model），而不是通用的 路由（Routing） 图。缓存 算子（Operators）、纹理/常量路径、异步复制（Asynchronous Copy） 机制、对等内存和特定于体系结构的功能可以更改路由。[1][3]重要的一点是失误意味着什么。如果 L1 无法提供所请求的扇区，则该请求会消耗下游的容量和带宽。如果 L2 也未命中，则 设备内存（Device Memory） 必须提供数据。因此，缓存命中（Cache Hit） 可以节省下游流量，并且通常可以避免与更远的距离相关的一些延迟。

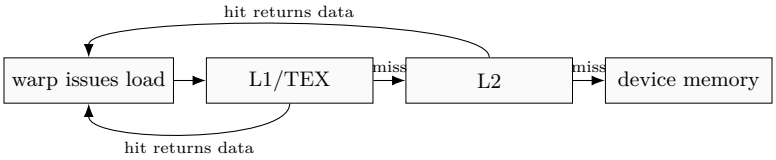
多个 线程束（Warps） 和多个请求可以同时进行。当依赖项等待时，调度程序可以发出不相关的工作。这就是 GPU 能够容忍大量访问 延迟（Latency） 而不减少请求本身的物理 延迟（Latency） 的方式。

因此，性能剖析器（Profiler） 需要的不仅仅是“负载计数”。 Nsight 公开 L1、L2 和 DRAM 接口的请求、扇区、命中率、字节和利用率。这些计数器有助于回答流量在哪里扩展、在哪里捕获重用以及哪个链接接近其 吞吐量（Throughput） 上限。 [3]

当 内核（Kernel） 速度缓慢时，请提出两个单独的问题：重要请求传输了多远？是否有足够的独立工作来重叠等待？

# 13 A load's journey through the hierarchy

For an ordinary cacheable global load on a recent NVIDIA GPU, a useful teaching path is:




This is a mental model, not a universal routing diagram. Cache operators, texture/constant paths, asynchronous copy mechanisms, peer memory, and architecture-specific features can change the route.[1][3]

The important point is what a miss means. If L1 cannot supply the requested sector, the request consumes capacity and bandwidth farther downstream. If L2 also misses, device memory must provide the data. A cache hit therefore saves downstream traffic and usually avoids some of the delay associated with going farther out.

Multiple warps and multiple requests can be in flight at the same time. The scheduler can issue unrelated work while a dependency waits. This is how a GPU can tolerate substantial access latency without reducing the physical latency of the request itself.

A profiler therefore needs more than "load count." Nsight exposes requests, sectors, hit rates, bytes, and utilization across L1, L2, and DRAM interfaces. Those counters help answer where traffic expanded, where reuse was captured, and which link approached its throughput ceiling.[3]

When a kernel is slow, ask two separate questions: *How far did the important requests travel?* and *Was there enough independent work to overlap the wait?*

# 14 时间局部性

对应原文第 18 页；点击查看相邻原文对照页

14 时间局部性 时间局部性意味着数据在最近被访问后会被重用，因此将其保留在附近可能会有回报。

考虑在短时间内由多个线程块使用的权重图块。第一个请求可能会从 设备内存（Device Memory） 获取它。如果该图块保留在 L2 中，则后面的块可以重用它，而无需再次读取 HBM。 张量（Tensor） 本身没有改变。只有可重用副本的位置有。

同样的原理也出现在较小的尺度上。加载到 寄存器（Register） 中的 值（Value） 可以参与许多算术指令。共享内存块可以供给许多线程。 缓存行（Cache Line） 在驱逐之前可以满足多次访问。

仅当重用在存储级别需要空间用于其他内容之前到达时，时间局部性才有用。这就是为什么 缓存容量（Cache Capacity） 和周围的访问流很重要。在接触大量不相关数据后重用一个 值（Value） 对于小型缓存来说可能已经太晚了，无法保留它。

推理时间重用的一个简单方法是问：

- 多久之后将再次需要此值（Value）？

- 在此之前有多少其他数据竞争同一存储？

- 重用对于硬件缓存策略是否可见，或者软件是否应该显式地暂存值（Value）？

时间局部性并不是“同一个地址出现两次”。它是在时间尺度和 工作集（Working Set） 上发生的重用，其中层次结构的某些级别可以实际保留数据。

AI 工作负载包含多个尺度的时间局部性：词元（Tokens） 和批处理元素之间的重复权重、内核（Kernel） 内重复使用的 矩阵（Matrix） 切片、跨位置共享的 归一化（Normalization） 参数以及跨解码步骤读取的 KV 状态。哪个级别捕获该局部性取决于重用的大小和时间。

# 14 Temporal locality

Temporal locality means data is reused after it was accessed recently enough that keeping it nearby may pay off.

Consider a weight tile used by several thread blocks over a short interval. The first request may fetch it from device memory. If the tile remains in L2, later blocks can reuse it without another HBM read. The tensor itself has not changed. Only the location of the reusable copy has.

The same principle appears at smaller scales. A value loaded into a register may participate in many arithmetic instructions. A shared-memory tile may feed many threads. A cache line may satisfy multiple accesses before eviction.

Temporal locality is useful only when the reuse arrives before the storage level needs the space for something else. This is why cache capacity and the surrounding access stream matter. Reusing one value after touching a huge amount of unrelated data may be too late for a small cache to retain it.

A simple way to reason about temporal reuse is to ask:

- How soon will this value be needed again?
- How much other data competes for the same storage before then?
- Is the reuse visible to hardware cache policy, or should software stage the value explicitly?

> Temporal locality is not “the same address appears twice.” It is reuse that occurs on a timescale and working set where some level of the hierarchy can realistically retain the data.

AI workloads contain temporal locality at many scales: repeated weights across tokens and batch elements, reused matrix tiles inside a kernel, normalization parameters shared across positions, and KV state read across decoding steps. Which level captures that locality depends on the size and timing of the reuse.

# 15 空间局部性

对应原文第 19 页；点击查看相邻原文对照页

15 空间局部性 空间局部性意味着附近的地址被足够紧密地一起访问，使得以块的形式获取数据是有用的。

缓存和内存接口通常不会为每个源级负载从 HBM 移动一个任意标量。它们传输对齐的扇区或更大的块。例如，当前的 Nsight 文档报告了 L1 和 L2 中的 32 字节扇区，而 L2 请求可能涵盖 128 字节 缓存行（Cache Line） 中的最多四个扇区。 [3]

如果 线程束（Warp） 的线程需要相邻的 4 字节 值（Values），这些有用的字节可以共享少量扇区。如果相同的 32 个线程接触的地址分散得很远，则即使源代码每个线程仍然包含一条加载指令，硬件也可能需要许多扇区。

因此，空间局部性影响比率 ηtraffic = 存储系统传输的程序字节请求的有用字节。

这不是一个通用的硬件指标，但它是一个有用的会计思想。比率低意味着移动花费在 线程束（Warp） 不使用的字节上。

空间局部性与时间局部性不同。 内核（Kernel） 可以一次传输完美连续的数据，提供出色的空间行为，但几乎没有时间重用。另一个 内核（Kernel） 可能会重复访问一小组分散的位置，从而在没有简单连续模式的情况下实现时间重用。

层次结构出于不同的原因奖励这两种局部性：空间局部性使每次传输更有用，而时间局部性使相同的传输数据在被驱逐之前再次有用。

合并访问（Coalescing） 是 GPU 特定的机制，可将 线程束（Warp） 的一组全局内存请求转换为访问模式允许的尽可能少的内存事务。

# 15 Spatial locality

Spatial locality means nearby addresses are accessed together closely enough that fetching data in blocks is useful.

Caches and memory interfaces do not generally move one arbitrary scalar from HBM for every source-level load. They transfer aligned sectors or larger blocks. Current Nsight documentation, for example, reports 32-byte sectors in L1 and L2, while L2 requests may cover up to four sectors from a 128-byte cache line.[3]

If a warp's threads need neighboring 4-byte values, those useful bytes can share a small number of sectors. If the same 32 threads touch addresses spread far apart, the hardware may need many sectors even though the source code still contains one load instruction per thread.

Spatial locality therefore affects the ratio

$$\eta_{\text{traffic}} = \frac{\text{useful bytes requested by the program}}{\text{bytes transferred by the memory system}}.$$

This is not a universal hardware metric, but it is a useful accounting idea. A low ratio means movement is being spent on bytes the warp does not use.

Spatial locality is different from temporal locality. A kernel may stream perfectly contiguous data once, giving excellent spatial behavior but almost no temporal reuse. Another kernel may repeatedly revisit a small set of scattered locations, giving temporal reuse without a simple contiguous pattern.

> The hierarchy rewards both kinds of locality for different reasons: spatial locality makes each transfer more useful, while temporal locality makes the same transferred data useful again before it is evicted.

Coalescing is the GPU-specific mechanism that turns a warp's set of global-memory requests into as few memory transactions as the access pattern allows.

# 16 工作组和复用距离

对应原文第 20 页；点击查看相邻原文对照页

16 工作集和重用距离 工作集是在执行的某个阶段需要保持方便可用的数据子集。

相关大小取决于问题：线程的实时标量、块的共享图块、流式多处理器（SM） 的活动数据或 内核（Kernel） 重复重用的全局区域都是不同规模的工作集。

当您将活动的 工作集（Working Set） 与可用于保留它的存储进行比较时，缓存行为变得更容易推理。

假设 内核（Kernel） 重复循环 8 MiB 数据，并且相关共享缓存有足够的有效空间用于该区域。重用可能会被很好地捕获。如果同一个 内核（Kernel） 循环数百兆字节，而许多其他客户端竞争缓存，则较大部分的请求将需要发送到更远的地方。

对于同一想法，重用距离是一个有用的直觉。在对同一 值（Value） 的两次访问之间，触及了多少不同的竞争数据？介入的占用空间越大，小缓存保留 值（Value） 的可能性就越小。确切的缓存行为还取决于映射、关联性、替换策略和并发流量，因此重用距离并不是一个完整的预测器。

这解释了为什么应该谨慎使用“适合 L2”。 40 MB 张量（Tensor） 和 50 MB L2 并不能保证近乎完美的重用。其他数据、缓存行映射和并发客户端也很重要。同样，如果热子集很小或者访问模式重复重用某些区域，则大于 L2 的 工作集（Working Set） 仍然可以获得许多命中。

容量比较提供了必要的线索，而不是缓存 性能（Performance） 证明。工作集大小、重用顺序、映射和竞争共同决定了实际捕获的重用量。

这也是为什么即使数学运算保持不变，性能（Performance） 也会随着 张量形状（Tensor Shape） 发生急剧变化：形状可以将活动的 工作集（Working Set） 移动到缓存容量阈值。

# 16 Working sets and reuse distance

A **working set** is the subset of data that needs to remain conveniently available over some phase of execution. The relevant size depends on the question: a thread's live scalars, a block's shared tile, an SM's active data, or a kernel's repeatedly reused global region are all working sets at different scales.

Cache behavior becomes easier to reason about when you compare the active working set with the storage available to retain it.

Suppose a kernel repeatedly cycles through 8 MiB of data and the relevant shared cache has ample effective space for that region. Reuse may be captured well. If the same kernel cycles through hundreds of megabytes while many other clients compete for the cache, a larger fraction of requests will need to go farther out.

**Reuse distance** is a useful intuition for the same idea. Between two accesses to the same value, how much distinct competing data is touched? The larger that intervening footprint, the less likely a small cache is to retain the value. Exact cache behavior also depends on mapping, associativity, replacement policy, and concurrent traffic, so reuse distance is not a complete predictor.

This explains why "fits in L2" should be used carefully. A 40 MB tensor and a 50 MB L2 do not guarantee near-perfect reuse. Other data, cache-line mapping, and simultaneous clients matter. Likewise, a working set larger than L2 can still achieve many hits if its hot subset is small or if access patterns repeatedly reuse some regions.

> A capacity comparison gives a necessary clue, not a cache-performance proof. Working-set size, reuse order, mapping, and competition together determine how much reuse is actually captured.

This is also why performance can change sharply with tensor shape even when the mathematical operation stays the same: the shape can move the active working set across a cache-capacity threshold.

# 17 缓存命中、未命中和下游流量

对应原文第 21 页；点击查看相邻原文对照页

17 缓存命中、未命中和下游流量 缓存命中（Cache Hit） 很有价值，因为它可以满足请求，而无需将相同的请求发送到更远的级别。

因此，主要的记忆结果是流量过滤。

如果 1 TB 的请求到达 L2 并且大部分都在那里得到服务，HBM 可能会看到远少于 1 TB 的读取流量。如果大多数请求未能满足，HBM 会看到更多的需求。 Nsight 的内存表专门公开了命中率和扇区缺失，以使这种区别可见。[3]

命中率本身仍然不完整。考虑两个 内核（Kernels）：

- 内核（Kernel） A 发出 1000 万次请求，命中率为 95%。

- 内核（Kernel） B 发出 100 万个请求，命中率为 80%。

内核（Kernel） A 具有更好的百分比，但从绝对值来看仍然可以向下游发送更多未命中。每个请求执行的有用工作量也很重要。

延迟（Latency） 行为也可能不同。一个缓存级别的命中可能比传播更远的未命中更快，但确切的损失取决于体系结构和请求类型。一些专门的缓存路径（例如纹理获取）具有不应推广到普通全局加载的语义。[1][3]评估缓存时，请查看下游保存的字节和请求，而不仅仅是命中率百分比。当从实际限制 内核（Kernel） 的路径中删除流量或 延迟（Latency） 时，高命中率非常有用。

这种框架对于数据量巨大的AI 内核（Kernels） 尤为重要。重用方面的适度改进可以消除千兆字节的 HBM 流量，而微小侧面结构上惊人的命中率百分比可能对 实际耗时（Wall-clock Time） 几乎没有影响。

# 17 Cache hits, misses, and downstream traffic

A cache hit is valuable because it can satisfy a request without sending the same request to a farther level. The primary memory consequence is therefore **traffic filtering**.

If 1 TB of requests arrive at L2 and a large fraction are served there, HBM may see far less than 1 TB of read traffic. If most requests miss, HBM sees much more of the demand. Nsight's memory tables expose hit rates and sector misses specifically to make this distinction visible.[3]

Hit rate by itself is still incomplete. Consider two kernels:

- Kernel A makes 10 million requests with a 95% hit rate.
- Kernel B makes 1 million requests with an 80% hit rate.

Kernel A has the better percentage but can still send more misses downstream in absolute terms. The amount of useful work performed per request also matters.

Latency behavior can differ too. A hit at one cache level may be faster than a miss that travels farther, but the exact penalty depends on architecture and request type. Some specialized cache paths, such as texture fetches, have semantics that should not be generalized to ordinary global loads.[1][3]

> When evaluating a cache, look at **bytes and requests saved downstream**, not only the hit-rate percentage. A high hit rate is useful when it removes traffic or latency from the path that was actually limiting the kernel.

This framing becomes especially important in AI kernels whose data volumes are enormous. A modest improvement in reuse can remove gigabytes of HBM traffic, while a spectacular hit-rate percentage on a tiny side structure may have almost no effect on wall-clock time.

# 18 合并访问（Coalescing）：组织 线程束（Warp） 的内存请求

对应原文第 22 页；点击查看相邻原文对照页

18 合并访问（Coalescing）：组织 线程束（Warp） 的内存请求 当 线程束（Warp） 中的线程发出全局内存指令时，GPU 将其地址组合到满足 线程束（Warp） 所需的内存事务中。好的 合并访问（Coalescing） 意味着这些事务包含线程实际使用的大部分字节。[1][2]当前 CUDA 示例 模型（Model） 32 字节事务中的全局内存服务。对于具体的 64 位情况，加载 32 对齐的连续 8 字节 值（Values） 的完整 线程束（Warp） 需要 256 个有用字节。这些地址占用 8 个 32 字节段。[1][3]车道编号不是规则。如果通道以不同的顺序访问相同的 32 个 值（Values），则所需段的集合可以保持不变。因此，合并访问（Coalescing） 与 线程束（Warp） 创建的地址占用有关，而不是强制 Lanei 读取 elementi。[1][2]可怜的 合并访问（Coalescing） 浪费了运动。如果这 32 个有用的 8 字节 值（Values） 被分散，以便每个都落在不同的 32 字节段中，则请求占用空间将增长到 32 个段，即使有用的有效负载仍然是 256 字节。

对齐很重要，因为跨越附加段的连续请求可能需要额外的事务。缓存行为可以减轻某些错位的成本，但它并不能消除分散的访问模式。[2]

合并访问（Coalescing） 是一个流量效率概念。它减少了 线程束（Warp） 请求所需的内存段数量。它不应该被简化为“线程必须始终访问连续的地址”。 AS07 将详细处理事务、对齐、跨步和特定于体系结构的合并访问（Coalescing） 规则。这里我们只需要了解为什么张量（Tensor）布局可以改变内存流量（Memory Traffic）而不改变张量（Tensor）大小。

# 18 Coalescing: organize a warp's memory requests

When threads in a warp issue a global-memory instruction, the GPU combines their addresses into the memory transactions needed to satisfy the warp. Good coalescing means those transactions contain a high fraction of bytes the threads actually use.[1][2]

Current CUDA examples model global-memory service in 32-byte transactions. For a concrete 64-bit case, a full warp loading 32 aligned consecutive 8-byte values needs 256 useful bytes. Those addresses occupy eight 32-byte segments.[1][3]

Lane numbering is not the rule. If the lanes access the same 32 values in a different order, the set of required segments can remain unchanged. Coalescing is therefore about the address footprint created by the warp, not about forcing lane $i$ to read element $i$.[1][2]

Poor coalescing wastes movement. If those 32 useful 8-byte values are spread so that each falls in a different 32-byte segment, the request footprint grows to 32 segments even though the useful payload is still 256 bytes.

Alignment matters because a contiguous request that straddles an additional segment can require an extra transaction. Cache behavior can soften the cost of some misalignment, but it does not make scattered access patterns free.[2]

**Coalescing is a traffic-efficiency concept.** It reduces the number of memory segments needed for a warp's request. It should not be simplified to "threads must always access consecutive addresses."

AS07 will treat transactions, alignment, stride, and architecture-specific coalescing rules in detail. Here we need only enough to see why tensor layout can change memory traffic without changing tensor size.

# 19 工作示例：连续负载和跨步负载

对应原文第 23 页；点击查看相邻原文对照页

19 工作示例：连续和跨步加载 假设一个 线程束（Warp） 有 32 个活动线程，每个线程加载一个对齐的 8 字节 值（Value），并且访问占用空间以 32 字节段计算。[1][3]情况A：跨步1.Thread i读取elementi。

地址占用 8 个 32 字节段，因此 256 个段字节覆盖 256 个有用字节。

情况B：跨步2.线程i读取元素2i。请求的值（Values）占用16个段。 线程束（Warp） 仍需要 256 个有用字节，而覆盖的段占用空间为 512 个字节。

情况 C：跨步 4.线程 i 读取元素 4i。每个请求的 8 字节 值（Value） 都以不同的 32 字节段开始，因此 32 段覆盖 1024 字节，共 256 个有用字节。

模式段 段字节 有用分数步幅 1 8 256 B 100% 步幅 2 16 512 B 50% 步幅 4 32 1024 B 25% 这些计数描述了此简化对齐示例中 线程束（Warp） 的段需求。如果请求命中缓存，则实际 HBM 流量可能会较低，并且特定于体系结构的路径可以包含其他详细信息。这个教训是关于请求几何的，而不是承诺每个段都成为 DRAM 事务。

在所有三种情况下，源代码可以包含相同数量的加载。内存系统不做同样多的工作。布局和步幅决定了传输的数据量是有用的。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 19 Worked example: contiguous and strided loads

Assume one warp has 32 active threads, each loads one aligned 8-byte value, and the access footprint is counted in 32-byte segments.[1][3]

**Case A: stride 1.** Thread $i$ reads element $i$.

$$32 \times 8 = 256 \text{ useful bytes.}$$

The addresses occupy eight 32-byte segments, so 256 segment bytes cover 256 useful bytes.

**Case B: stride 2.** Thread $i$ reads element $2i$. The requested values occupy 16 segments. The warp still needs 256 useful bytes, while the covered segment footprint is 512 bytes.

**Case C: stride 4.** Thread $i$ reads element $4i$. Each requested 8-byte value begins in a different 32-byte segment, so 32 segments cover 1024 bytes for 256 useful bytes.

| Pattern | Segments | Segment bytes | Useful fraction |
|---|---:|---:|---:|
| stride 1 | 8 | 256 B | 100% |
| stride 2 | 16 | 512 B | 50% |
| stride 4 | 32 | 1024 B | 25% |

These counts describe the warp's segment demand in this simplified aligned example. Actual HBM traffic can be lower if requests hit in cache, and architecture-specific paths can have additional details. The lesson is about request geometry, not a promise that every segment becomes a DRAM transaction.

The source code can contain the same number of loads in all three cases. The memory system does not do the same amount of work. Layout and stride determine how much transferred data is useful.

# 20 分块（Tiling） 将容量转化为再利用

对应原文第 24 页；点击查看相邻原文对照页

20 分块（Tiling） 将容量转化为重用 分块（Tiling） 将大问题分成足够小的部分，以便在重用时保留在更近的存储级别中。

考虑一个玩具 通用矩阵乘法（GEMM），其中一个块负责 C =AB 的 64× 64 输出图块，全 K 维度为 32。对于 BF16 输入，该块需要 64× 32 的 A 图块和 32× 64 的 B 图块：

这 8 KiB 输入支持

浮点（Floating-point） 乘法和加法时的运算是分开计算的。

如果输出也是 BF16 并且β = 0，则写入最终的 64×64 结果会再增加 8 KiB。在这种简化的计算下，玩具问题对于 16 KiB 的输入/输出移动执行 262,144 浮点运算次数（FLOPs），或者在该边界处每个字节执行 16 浮点运算次数（FLOPs）。

重要的部分是重用。 A 瓦片的每个元素贡献许多输出列，B 瓦片的每个元素贡献许多输出行。在 K >32 的较大 通用矩阵乘法（GEMM） 中，一个块在多个 K 切片上重复此分级，同时部分输出累加器仍接近计算。

HBM 切片共享内存切片 寄存器（Register） 片段每次重复使用时加载一次 分块（Tiling） 不会使 张量（Tensor） 变小。它更改了计划，以便较小的 工作集（Working Set） 在离开快速存储级别之前可以重复使用。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 20 Tiling turns capacity into reuse

Tiling divides a large problem into pieces small enough to stay in a closer storage level while they are reused.

Consider a toy GEMM in which one block is responsible for a $64 \times 64$ output tile of $C = AB$ and the full $K$ dimension is 32. With BF16 inputs, the block needs an $A$ tile of $64 \times 32$ and a $B$ tile of $32 \times 64$:

$$(64 \times 32 + 32 \times 64) \times 2 = 8192 \text{ bytes} = 8 \text{ KiB}.$$

Those 8 KiB of inputs support

$$2 \times 64 \times 64 \times 32 = 262{,}144$$

floating-point operations when a multiply and add are counted separately.

If the output is also BF16 and $\beta = 0$, writing the final $64 \times 64$ result adds another 8 KiB. Under this simplified accounting, the toy problem performs 262,144 FLOPs for 16 KiB of input/output movement, or 16 FLOPs per byte at that boundary.

The important part is the reuse. Each element of the $A$ tile contributes to many output columns, and each element of $B$ contributes to many output rows. In a larger GEMM with $K > 32$, a block repeats this staging across multiple $K$ slices while partial output accumulators remain close to compute.




Tiling does not make the tensor smaller. It changes the schedule so that a smaller working set can be reused before it leaves a fast storage level.

# 21 通用矩阵乘法（GEMM） 也是一个内存重用问题

对应原文第 25 页；点击查看相邻原文对照页

21 通用矩阵乘法（GEMM） 也是一个内存重用问题 矩阵乘法（Matrix Multiplication） 因其算术 吞吐量（Throughput） 而闻名，但高 性能（Performance） 通用矩阵乘法（GEMM） 也是一个精心设计的数据移动时间表。

简单的循环可以从 设备内存（Device Memory） 中重复获取相同的 矩阵（Matrix） 元素。相反，阻塞实现将图块映射到线程块，在芯片上暂存可重用输入，将较小的片段加载到 寄存器（Registers） 中，并在写回结果之前累积许多结果。

CUTLASS 准确地描述了这种层次结构：线程块提供粗略并行性，共享内存块保留用于协作 线程束（Warps） 的数据，而 寄存器（Register） 片段则为重复的 矩阵（Matrix） 指令保留 值（Values）。 [14]

这就是为什么通用矩阵乘法（GEMM）可以达到很高的算术强度（Arithmetic Intensity）。相同的字节被故意重复使用多次。然而，随着 矩阵（Matrix） 尺寸和瓷砖形状的变化，重复使用可能会减弱。小 矩阵（Matrices）、窄尺寸、笨拙的尾部或平行度不足可能无法像大型形状良好的 通用矩阵乘法（GEMM）那样有效地摊销运动。

内存层次结构（Memory Hierarchy） 还造成资源权衡。较大的图块可以增加重复使用，但会消耗更多的共享内存（Shared Memory） 和 寄存器（Registers）。更深层次的软件管道可以与计算重叠传输，但需要额外的缓冲。如果 占用率（Occupancy） 太少或产生库冲突，最大化重用的设计仍然会失败。

通用矩阵乘法（GEMM） 的快速部分不仅仅是“张量核心（Tensor Cores） 很快”。高效的 通用矩阵乘法（GEMM） 安排了 数据移动（Data Movement），因此昂贵的 矩阵（Matrix） 硬件会从附近的存储中连续馈送，而每个获取的图块都会重复使用多次。

这是通往后面系统主题的桥梁。 AS03 中的 算术强度（Arithmetic Intensity） 量化每个字节的工作量。 AS04 中的 Roofline 将这种强度与硬件天花板联系起来。共享内存和 寄存器（Register） 手册解释了使重用成为可能的片上暂存细节。

# 21 GEMM is a memory-reuse problem too

Matrix multiplication is famous for its arithmetic throughput, but high-performance GEMM is also a carefully engineered data-movement schedule.

A naive loop can repeatedly fetch the same matrix elements from device memory. A blocked implementation instead maps tiles to thread blocks, stages reusable inputs on chip, loads smaller fragments into registers, and accumulates many products before writing results back.

CUTLASS describes exactly this hierarchy: thread blocks provide coarse parallelism, shared-memory tiles retain data for cooperating warps, and register fragments retain values for repeated matrix instructions.[14]

This is why GEMM can reach high arithmetic intensity. The same bytes are deliberately reused many times. As matrix dimensions and tile shapes change, however, that reuse can weaken. Small matrices, skinny dimensions, awkward tails, or insufficient parallelism may fail to amortize movement as effectively as a large well-shaped GEMM.

The memory hierarchy also creates resource trade-offs. Larger tiles can increase reuse but consume more shared memory and registers. Deeper software pipelines can overlap transfers with compute but require additional buffering. A design that maximizes reuse can still lose if it leaves too little occupancy or creates bank conflicts.

> The fast part of GEMM is not merely “Tensor Cores are fast.” Efficient GEMM arranges data movement so expensive matrix hardware is continuously fed from nearby storage while each fetched tile is reused many times.

This is the bridge to later systems topics. Arithmetic intensity in AS03 quantifies work per byte. Roofline in AS04 relates that intensity to hardware ceilings. Shared-memory and register handbooks explain the on-chip staging details that make the reuse possible.

# 移动 22 字节与执行算术运算

对应原文第 26 页；点击查看相邻原文对照页

移动 22 个字节与执行算术运算 内核（Kernel） 的数学运算计数并不能告诉您它需要多少 数据移动（Data Movement）。有用的第一个比率是 算术强度（Arithmetic Intensity）= 浮点运算次数（FLOPs） 在所选内存边界移动的字节。

边界很重要。每个 HBM 字节、每个 L2 字节和每个 L1 字节的 浮点运算次数（FLOPs） 可能不同，因为缓存和片上重用过滤流量。 AS03会谨慎对待这一点。

考虑 32 位浮点（FP32） 向量（Vector） 添加：

忽略缓存重用并假设对 A 进行一次读取、对 B 进行一次读取、对 C 进行一次写入，对于一次 浮点（Floating-point） 加法，每个元素移动 12 个字节：

1 浮点运算（FLOP）

几乎没有重复使用来放大每个字节的 算术工作量（Arithmetic Work）。

前几页中的平铺 通用矩阵乘法（GEMM） 示例非常不同。在其简化的输入加输出计算下，一个小块已经达到每字节 16 浮点运算次数（FLOPs），因为每个输入 值（Value） 在替换之前参与了许多乘加。

这并不能证明 向量（Vector） 加法始终是 带宽受限（Bandwidth-bound） 或 通用矩阵乘法（GEMM） 始终是 计算受限（Compute-bound）。实际的 性能（Performance） 取决于形状、启动开销（Launch Overhead）、缓存状态、精度（Precision）、硬件和其他瓶颈。

该比率简单地揭示了可用于摊销 数据移动（Data Movement） 的算术数量。

如果第一个工作负载移动字节效率低下，而第二个工作负载重用数据良好，则较低的 浮点运算次数（FLOP Count） 可能会比较高的 浮点运算次数（FLOP Count） 运行得慢。 浮点运算次数（FLOPs）和流量为独立核算维度。

屋顶线模型（Roofline Model） 形式化了这种关系，但 AS02 只需要直觉：AI 性能（Performance） 取决于数学量和必须跨越约束接口的字节数。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 22 Bytes moved versus arithmetic performed

A kernel's mathematical operation count does not tell you how much data movement it requires. A useful first ratio is

$$\text{arithmetic intensity} = \frac{\text{FLOPs}}{\text{bytes moved at a chosen memory boundary}}.$$

The boundary matters. FLOPs per HBM byte, per L2 byte, and per L1 byte can be different because caches and on-chip reuse filter traffic. AS03 will treat this carefully.

Consider FP32 vector addition:

$$C_i = A_i + B_i.$$

Ignoring cache reuse and assuming one read of $A$, one read of $B$, and one write of $C$, each element moves 12 bytes for one floating-point addition:

$$\frac{1 \text{ FLOP}}{12 \text{ bytes}} \approx 0.083 \text{ FLOP/byte}.$$

There is little reuse to amplify the arithmetic work per byte.

The tiled GEMM example from the previous pages was very different. Under its simplified input-plus-output accounting, a small tile already reached 16 FLOPs per byte because each input value participated in many multiply-adds before replacement.

This does not prove that vector addition is always bandwidth-bound or GEMM is always compute-bound. Actual performance depends on shape, launch overhead, cache state, precision, hardware, and other bottlenecks. The ratio simply reveals how much arithmetic is available to amortize data movement.

A lower FLOP count can run slower than a higher FLOP count if the first workload moves bytes inefficiently and the second reuses data well. FLOPs and traffic are independent accounting dimensions.

The Roofline model formalizes this relationship, but AS02 needs only the intuition: AI performance depends on both the amount of math and the number of bytes that must cross constrained interfaces.

# 23 为什么元素 内核（Kernels） 和 通用矩阵乘法（GEMM） 的行为不同

对应原文第 27 页；点击查看相邻原文对照页

23 为什么按元素 内核（Kernels） 和 通用矩阵乘法（GEMM） 的行为不同 按元素操作通常会触及较大的张量（Tensor），对每个元素执行少量算术，然后继续。 矩阵乘法（Matrix Multiplication） 重复组合相同的输入图块以产生许多输出。这两种结构对 内存层次结构（Memory Hierarchy） 提出了截然不同的要求。

对于元素 激活（Activation），值（Value） 可以读取一次、转换并写入一次。在 值（Value） 离开 内核（Kernel） 之前，重复使用该 值（Value） 的机会有限。如果算术便宜，则读写 张量（Tensor） 的成本可能占主导地位。

对于 通用矩阵乘法（GEMM），阻止会导致故意重用。 A 的图块可以参与许多输出列，而 B 的图块可以参与许多输出行。因此，相同的加载字节可以提供更多的算术。

这种差异解释了为什么融合很重要。假设 激活（Activation） 从 HBM 读取大的 通用矩阵乘法（GEMM） 输出，并写回另一个 张量（Tensor）。如果 激活（Activation） 可以作为 通用矩阵乘法（GEMM） 尾声的一部分执行，而结果仍在附近，则大型中间体的一次往返可能会消失。 CUTLASS 明确将尾声视为 缩放（Scaling） 和元素工作可以与 矩阵（Matrix） 结果组合的地方。 [14]

融合不会自动带来好处。它可能会增加 寄存器压力（Register Pressure）、使调度复杂化、减少其他地方的重用或阻止使用高度优化的库 内核（Kernel）。 AS11 将处理这些权衡。

记忆问题不是“这个操作简单吗？”它是“每个级别必须跨越多少字节才能执行有用的工作？”当单行元素公式应用于数十亿个 值（Values） 且很少重复使用时，其成本可能会很高。

# 23 Why elementwise kernels and GEMM behave differently

Elementwise operations often touch a large tensor, perform a small amount of arithmetic per element, and move on. Matrix multiplication repeatedly combines the same input tiles to produce many outputs. Those two structures place very different demands on the memory hierarchy.

For an elementwise activation, a value may be read once, transformed, and written once. There is limited opportunity to reuse that value before it leaves the kernel. If the arithmetic is cheap, the cost of reading and writing the tensor can dominate.

For GEMM, blocking creates deliberate reuse. A tile of $A$ can participate in many output columns, while a tile of $B$ participates in many output rows. The same loaded bytes therefore feed far more arithmetic.

This difference explains why fusion can matter. Suppose an activation reads a large GEMM output from HBM and writes another tensor back. If the activation can be performed as part of the GEMM epilogue while results are still nearby, one round trip of a large intermediate may disappear. CUTLASS explicitly treats epilogues as a place where scaling and elementwise work can be combined with the matrix result.[14]

Fusion is not automatically beneficial. It can increase register pressure, complicate scheduling, reduce reuse elsewhere, or prevent use of a highly optimized library kernel. AS11 will treat those trade-offs.

> The memory question is not “Is this operation simple?” It is “How many bytes must cross each level for the useful work performed?” A one-line elementwise formula can be expensive when applied to billions of values with little reuse.

# 24 注意力机制（Attention） 也是一个数据移动问题

对应原文第 28 页；点击查看相邻原文对照页

24 注意力机制（Attention） 也是一个数据移动问题 稠密注意力（Dense Attention） 结合了 矩阵乘法（Matrix Multiplication）、Softmax 归一化（Softmax）、屏蔽和另一个 矩阵乘法（Matrix Multiplication）。熟悉的等式

没有指定中间分数和概率 值（Values） 必须存在的位置。

简单的分解可以实现 N×N 分数矩阵（Score Matrix），读取它以进行 softmax，写入概率，然后再次读取它们以进行 PV 乘法。对于长序列，这些中间体足够大，以至于通过 设备内存（Device Memory） 移动它们可能会成为主要成本。

内存层次结构（Memory Hierarchy） 开启了另一种可能性：形成更小的分数块，在它们保留在芯片上时消耗它们，仅维护 合并（Combine） Softmax 归一化（Softmax） 块所需的状态，并避免将全密集中间体写入 HBM。

这是计划更改，而不是对 稠密注意力（Dense Attention） 函数的更改。算术交互仍然密集。该实现只需选择一个内存调度，该调度可以更长时间地保留接近计算的有用中间状态。

方程指定数据依赖性，而不是存储决策。实现可以生成中间图块并立即使用它，而无需在 设备内存（Device Memory） 中分配完整的中间 张量（Tensor）。

这个想法比 注意力机制（Attention） 更广泛。 内核（Kernel） 融合、重计算（Recomputation）、分块（Tiling） 和 算子（Operator） 调度都利用相同的原理：有时额外的本地工作比将大型中间体移出到 HBM 并返回要便宜。

FlashAttention 是典型的 AI 示例，但确切的算法属于 Understanding AI 系列。

这里我们用它只是为了揭示内存层次结构的教训。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 24 Attention is also a data-movement problem

Dense attention combines matrix multiplication, softmax, masking, and another matrix multiplication. The familiar equation

$$O = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$$

does not specify where the intermediate score and probability values must live.

A straightforward decomposition can materialize an $N \times N$ score matrix, read it for softmax, write probabilities, and read them again for the $PV$ multiplication. For long sequences, those intermediates are large enough that moving them through device memory can become a major cost.

The memory hierarchy opens another possibility: form smaller score tiles, consume them while they remain on chip, maintain only the state needed to combine softmax blocks, and avoid writing the full dense intermediate to HBM.

That is a scheduling change, not a change to the dense attention function. The arithmetic interactions remain dense. The implementation simply chooses a memory schedule that retains useful intermediate state close to compute for longer.

> Equations specify data dependencies, not storage decisions. An implementation can generate an intermediate tile and consume it immediately without ever allocating the complete intermediate tensor in device memory.

This idea is broader than attention. Kernel fusion, recomputation, tiling, and operator scheduling all exploit the same principle: sometimes extra local work is cheaper than moving a large intermediate out to HBM and back.

FlashAttention is the canonical AI example, but the exact algorithm belongs to the Understanding AI series. Here we use it only to expose the memory-hierarchy lesson.

# 25 FlashAttention 作为层次结构感知示例

对应原文第 29 页；点击查看相邻原文对照页

25 FlashAttention 作为层次结构感知示例 Dense FlashAttention 避免将完整分数和概率 矩阵（Matrices） 转换为 HBM-驻留（Resident） 中间体。生成分数图块，在其仍在芯片上时将其合并到按行 归一化（Normalization） 和输出状态中，然后将其丢弃。正在评估的密集 Softmax 归一化（Softmax）-注意力机制（Attention） 函数未更改。 [13]

重要的记忆转换是：

在 HBM 片上图块 + 运行状态输出加载图块更新中，其 IO 分析将 HBM 和有限片上存储之间的移动视为显式算法成本，并得出比论文内存 模型（Model） 下的标准物化计划更少的 HBM 访问。 [13]

这个例子纠正了两个常见的误解。

首先，减少 HBM 流量并不需要减少 浮点运算次数（FLOPs）。 FlashAttention 甚至可以在反向传播中重新计算一些中间产物，因为额外的算术可能比存储和重新读取大张量更便宜。

其次，“注意力机制（Attention） 是 内存受限（Memory-bound）”过于宽泛。主要瓶颈取决于序列长度、头维度（Head Dimension）、精度（Precision）、硬件、前向与后向以及 预填充（Prefill） 与解码。 内存层次结构（Memory Hierarchy） 告诉我们哪里会产生成本。它不允许我们为每个 注意力机制（Attention） 工作负载标记一个瓶颈。

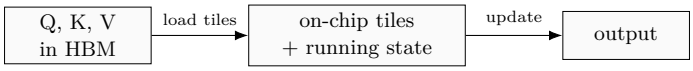
持久的教训不是“使用 FlashAttention”。就是算法设计可以包含内存层次结构（Memory Hierarchy）。中间体的创建、重用和废弃位置与定义它的方程式一样重要。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 25 FlashAttention as a hierarchy-aware example

Dense FlashAttention avoids making the complete score and probability matrices into HBM-resident intermediates. A score tile is generated, merged into the row-wise normalization and output state while it is still on chip, and then discarded. The dense softmax-attention function being evaluated is unchanged.[13]

The important memory transformation is:




Its IO analysis treats movement between HBM and limited on-chip storage as an explicit algorithmic cost and derives fewer HBM accesses than a standard materializing schedule under the paper's memory model.[13]

This example corrects two common misunderstandings.

First, reducing HBM traffic does not require reducing FLOPs. FlashAttention can even recompute some intermediates in backward propagation because extra arithmetic may be cheaper than storing and rereading large tensors.

Second, "attention is memory-bound" is too broad. The dominant bottleneck depends on sequence length, head dimension, precision, hardware, forward versus backward, and prefill versus decode. A memory hierarchy tells us where costs can arise. It does not let us label every attention workload with one bottleneck.

The durable lesson is not "use FlashAttention." It is that algorithm design can include the memory hierarchy. Where an intermediate is created, reused, and discarded can matter as much as the equation that defines it.

# 26 预填充（Prefill） 和解码压力存储器不同

对应原文第 30 页；点击查看相邻原文对照页

26 预填充（Prefill） 和解码压力存储器不同，LLM 推理（Inference） 包含至少两个不同的执行机制。

预填充（Prefill） 将多个提示符词元（Tokens） 一起处理。 矩阵（Matrix） 维度更大，有许多 查询（Query） 行，优化的 GEMM/注意力机制（Attention） 内核（Kernels） 可以暴露大量的重用和并行性。计算通常有更多机会在许多操作中摊销重量和 激活（Activation） 移动。

解码通常一次将每个序列推进一个新的 词元（Token）。在小批量大小下，许多层操作会变成 矩阵（Matrix）-向量（Vector） 或非常小的 矩阵（Matrix）-矩阵（Matrix） 形状。相同的 模型（Model） 权重仍然需要参与，但每个 词元（Token） 摊销这些读取的工作要少得多。不断增长的键值缓存（KV Cache）也必须由注意力机制（Attention）来阅读。

一个简单的下限思想实验说明了为什么 内存带宽（Memory Bandwidth） 很重要。考虑一个假设的密集 70 亿参数 模型（Model），每个权重存储 2 个字节。其权重以十进制单位计算约为 14 GB。如果每个 词元（Token） 都需要一次全权重扫描，并且 H100 SXM 除了这些读取之外什么都可以维持其宣传的 3.35 TB/s，那么仅扫描一项就需要

在计算 KV 流量、激活（Activations）、非权重读取、同步（Synchronization） 或不完美的带宽使用之前，对应于约 239 词元（Token）/s。 [7]

这是上限式健全性检查，而不是 吞吐量（Throughput） 预测。

预填充（Prefill） 和解码使用相同的 模型（Model），但在计算与内存交换空间中占据非常不同的点。在未测量实际形状和批次状态的情况下，切勿将“内存受限（Memory-bound）”或“计算受限（Compute-bound）”标签从一相转移到另一相。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 26 Prefill and decode stress memory differently

LLM inference contains at least two distinct execution regimes.

**Prefill** processes many prompt tokens together. Matrix dimensions are larger, there are many query rows, and optimized GEMMs/attention kernels can expose substantial reuse and parallelism. The computation often has more opportunity to amortize weight and activation movement across many operations.

**Decode** usually advances each sequence by one new token at a time. At small batch sizes, many layer operations become matrix-vector or very small matrix-matrix shapes. The same model weights still need to participate, but there is much less work per token to amortize those reads. The growing KV cache also has to be read by attention.

A simple lower-bound thought experiment shows why memory bandwidth matters. Consider a hypothetical dense 7-billion-parameter model stored at 2 bytes per weight. Its weights occupy about 14 GB in decimal units. If every token required one full weight sweep and an H100 SXM could sustain its advertised 3.35 TB/s on nothing but those reads, the sweep alone would take

$$14\ \text{GB}/3.35\ \text{TB/s} \approx 4.18\ \text{ms},$$

corresponding to about 239 token/s before counting KV traffic, activations, non-weight reads, synchronization, or imperfect bandwidth use.[7]

That is an upper-bound-style sanity check, not a throughput prediction.

> Prefill and decode use the same model but can occupy very different points in the compute-versus-memory trade space. Never transfer a “memory-bound” or “compute-bound” label from one phase to the other without measuring the actual shape and batch regime.

# 27个权重、激活（Activations）和KV状态有不同的寿命

对应原文第 31 页；点击查看相邻原文对照页

27 权重、激活（Activations） 和 KV 状态具有不同的生命周期 AI 系统会移动几个主要类别的数据，它们的生命周期决定了 内存层次结构（Memory Hierarchy） 如何提供帮助。

权重在请求和 词元（Tokens） 之间保持不变。层对许多批处理元素、序列位置和未来请求重用相同的参数。然而，大型 模型（Models） 远大于片上缓存，因此随着时间的推移，大部分权重流仍然来自 设备内存（Device Memory）。

激活是在 模型（Model） 执行时产生的。它们的生命周期取决于 训练（Training） 与 推理（Inference）、融合、检查点和 算子（Operator） 图。一些中间体会立即被消耗，而另一些则必须存活更长时间。

KV 缓存在自回归解码中持续存在，因为未来的 词元（Tokens） 会从较早的位置处理 键（Keys） 和 值（Values）。它的占用空间随着普通缓存 注意力机制（Attention） 中的序列长度而增长，因此长上下文解码可能会产生大量的持久内存需求和读取流量。

这些类别的重用粒度也有所不同。权重图块可以在一个 内核（Kernel） 内的一批中在多个 词元（Tokens） 中重复使用。 激活（Activation） 瓷砖只能在融合的 算子（Operator） 内重复使用。在长会话中，每个解码步骤可以重新访问 KV 块一次。

内存优化从生命周期开始。不仅要问“这个 张量（Tensor） 有多大？”而是“谁会重复使用它，多久，以及在什么范围内？”对于短命的 激活（Activation） 切片来说，不可变权重的正确存储策略可能是完全错误的。

这种分类还解释了为什么内存容量和带宽压力会随着工作负载而变化。 训练（Training） 存储更多状态。长上下文 推理（Inference） 增长 KV 状态。小批量解码可能会以有限的算术重用来重复流式传输权重。一个层次结构为所有这些服务提供服务，但流量模式不同。

# 27 Weights, activations, and KV state have different lifetimes

AI systems move several major classes of data, and their lifetimes shape how the memory hierarchy can help.

**Weights** are persistent across requests and tokens. A layer reuses the same parameters for many batch elements, sequence positions, and future requests. Large models, however, are far bigger than on-chip caches, so much of the weight stream still comes from device memory over time.

**Activations** are produced as the model executes. Their lifetime depends on training versus inference, fusion, checkpointing, and the operator graph. Some intermediates are consumed immediately, while others must survive much longer.

**KV cache** persists across autoregressive decoding because future tokens attend to keys and values from earlier positions. Its footprint grows with sequence length in ordinary cached attention, so long-context decode can create substantial persistent memory demand and read traffic.

These categories also differ in reuse granularity. A weight tile may be reused across many tokens in a batch inside one kernel. An activation tile may be reused only within a fused operator. A KV block may be revisited once per decode step across a long session.

> Memory optimization starts with lifetime. Ask not only “How big is this tensor?” but “Who will reuse it, how soon, and at what scope?” The right storage strategy for immutable weights can be completely wrong for short-lived activation tiles.

This classification also explains why memory capacity and bandwidth pressure shift with workload. Training stores more state. Long-context inference grows KV state. Small-batch decode may repeatedly stream weights with limited arithmetic reuse. One hierarchy serves all of them, but the traffic pattern is different.

# 28 训练（Training） 和 推理（Inference） 移动不同的字节

对应原文第 32 页；点击查看相邻原文对照页

28 训练（Training） 和 推理（Inference） 移动不同的字节 训练（Training） 和 推理（Inference） 执行相关的 模型（Model） 方程，但它们的内存行为不一样。

在 训练（Training） 期间，前向传播（Forward Pass） 生成反向传播所需的 激活（Activations）。向后计算梯度并可以读取保存的 激活（Activations） 或重新计算它们。优化器维护附加参数关联状态。因此，该系统移动和存储的重量远远超过 模型（Model） 单独的重量。

激活（Activation） 检查点通过存储更少的 激活（Activations） 并在向后过程中重新计算选定的向前工作来改变这种平衡。这会用额外的算术换取较低的 激活（Activation） 容量，这是用计算换取内存的一个明确示例。 AS23 将深入介绍该技术。

推理（Inference） 删除了 梯度（Gradient） 和优化器状态流量，但它引入了自己的机制。 预填充（Prefill） 一次处理多个 词元（Tokens）。自回归解码跨步骤保留 KV 状态，并且在小批量大小下，每个生成的 词元（Token） 的大权重 矩阵（Matrices） 的重用可能有限。

因此，相同的优化可以有不同的 值（Value）。在 训练（Training） 中保存 激活（Activation） 写入可能非常重要。在长上下文解码中，减少持久 KV 读取可能更重要。增加 批大小（Batch Size） 可以改善 推理（Inference） 中的权重重用，但可能会增加在线服务中的排队 延迟（Latency）。

请勿使用称为“模型（Model） 内存”的一个数字来描述 训练（Training） 和 推理（Inference）。权重、梯度、优化器状态、激活（Activations）、临时工作区和 键值缓存（KV Cache） 具有不同的生命周期和流量模式。

内存层次结构（Memory Hierarchy） 是这些差异背后的硬件基础。分片、检查点、卸载、连续批处理（Continuous Batching） 和 KV 分页等系统级技术决定哪些字节出现以及何时出现。 内核（Kernels） 决定每个活动 工作集（Working Set） 在 GPU 中移动的效率。

# 28 Training and inference move different bytes

Training and inference execute related model equations, but their memory behavior is not the same.

During training, the forward pass produces activations needed by backward propagation. Backward computes gradients and may read saved activations or recompute them. Optimizers maintain additional parameter-associated state. The system therefore moves and stores far more than model weights alone.

Activation checkpointing changes this balance by storing fewer activations and recomputing selected forward work during backward. That trades extra arithmetic for lower activation capacity, an explicit example of exchanging compute for memory. AS23 will cover the technique in depth.

Inference removes gradient and optimizer-state traffic, but it introduces its own regimes. Prefill processes many tokens at once. Autoregressive decode retains KV state across steps, and at small batch sizes may have limited reuse of large weight matrices per generated token.

The same optimization can therefore have different value. Saving an activation write may matter enormously in training. Reducing persistent KV reads may matter more in long-context decode. Increasing batch size can improve weight reuse in inference but may increase queueing latency in an online service.

> Do not use one number called “model memory” to describe both training and inference. Weights, gradients, optimizer state, activations, temporary workspaces, and KV cache have different lifetimes and traffic patterns.

The memory hierarchy is the hardware substrate beneath those differences. System-level techniques such as sharding, checkpointing, offloading, continuous batching, and KV paging decide which bytes are present and when. Kernels decide how efficiently each active working set moves through the GPU.

# 29 容量和带宽是独立的硬件属性

对应原文第 33 页；点击查看相邻原文对照页

29 容量和带宽是独立的硬件属性 当前的加速器使区别更加具体。下表使用供应商公布的每 GPU 容量和峰值 内存带宽（Memory Bandwidth），时间范围以引用的产品文档为准。这不是 性能（Performance） 排名。

GPU 内存类型 容量 峰值带宽 NVIDIA H100 SXM HBM3 80 GB 3.35 TB/s[7] NVIDIA H200 SXM HBM3e 141 GB 4.8 TB/s[8] NVIDIA B200 SXM HBM3e 180 GB 最高 8 TB/s[9] AMD MI300X HBM3 192 GB 5.3 TB/s[10] AMD MI355X HBM3E 288 GB 8 TB/s[11] 规格本身说明了为什么两列必须分开。从 H100 SXM 到 H200 SXM，公布的内存容量标签从 80 GB 上升到 141 GB，约增加了 76%，而峰值 内存带宽（Memory Bandwidth） 从 3.35 TB/s 上升到 4.8 TB/s，约增加了 43%。容量和带宽之间没有固定的换算。它们是独立的硬件属性，对工作负载具有不同的影响。[7][8]容量回答“工作集（Working Set） 可以驻留在设备上吗？”带宽回答“流量能够多快维持？”一个好的系统需要两者都足以满足其工作负载。

# 29 Capacity and bandwidth are independent hardware properties

Current accelerators make the distinction concrete. The table below uses vendor-advertised per-GPU capacity and peak memory bandwidth, time-scoped to the cited product documentation. It is not a performance ranking.

| GPU | Memory type | Capacity | Peak bandwidth |
|---|---|---|---|
| NVIDIA H100 SXM | HBM3 | 80 GB | 3.35 TB/s[7] |
| NVIDIA H200 SXM | HBM3e | 141 GB | 4.8 TB/s[8] |
| NVIDIA B200 SXM | HBM3e | 180 GB | up to 8 TB/s[9] |
| AMD MI300X | HBM3 | 192 GB | 5.3 TB/s[10] |
| AMD MI355X | HBM3E | 288 GB | 8 TB/s[11] |

The specifications themselves show why the two columns must be kept separate. From H100 SXM to H200 SXM, the published memory-capacity label rises from 80 GB to 141 GB, about 76%, while peak memory bandwidth rises from 3.35 TB/s to 4.8 TB/s, about 43%. There is no fixed conversion between capacity and bandwidth. They are independent hardware properties with different consequences for a workload.[7][8]

Capacity answers “can the working set reside on the device?” Bandwidth answers “how quickly can traffic be sustained?” A good system needs enough of both for its workload.

# 30 理论带宽与实际带宽

对应原文第 34 页；点击查看相邻原文对照页

30 理论带宽与实际带宽 供应商带宽规格描述了峰值硬件功能。 内核（Kernel） 所达到的速率必须根据一段时间内实际穿过接口的字节来测量。

CUDA 最佳实践定义了以下形式的有效带宽计算：[2] Beff = Bread +Bwrite

分子必须使用明确的字节计数约定，并且单位必须与用于比较的峰值数相匹配。

假设一个 内核（Kernel） 在 5 毫秒内读取 4 GiB 并写入 4 GiB。这是 8 GiB 的有用应用程序流量。以十进制字节每秒为单位，

这仍然没有告诉我们确切的 DRAM 流量。缓存、写入策略、ECC 行为、压缩和额外事务可能会使物理移动与有用字节计数不同。

因此，Nsight 在 L1、L2 和 设备内存（Device Memory） 处报告字节和 吞吐量（Throughput） 的硬件计数器，而不是要求开发人员从 张量（Tensor） 大小推断所有内容。 [3]

三个数字都可以不同：供应商峰值带宽、计算出的有用有效带宽

你的意思是。

这也是为什么跨纸带宽数字需要谨慎。不同的精度、张量（Tensor） 形状、缓存状态、时钟、ECC 设置和字节计数约定可以使表面上相似的 GB/s 数字变得无法比较。

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# 30 Theoretical bandwidth versus achieved bandwidth

Vendor bandwidth specifications describe a peak hardware capability. A kernel's achieved rate must be measured from the bytes that actually cross the interface over time.

CUDA Best Practices defines an effective-bandwidth calculation of the form[2]

$$B_{\text{eff}} = \frac{B_{\text{read}} + B_{\text{write}}}{t}.$$

The numerator must use a clear byte-counting convention, and the units must match the peak number being used for comparison.

Consider a hypothetical kernel that reads 4 GiB and writes 4 GiB in 5 ms. That is 8 GiB of useful application traffic. In decimal bytes per second,

$$\frac{8 \times 2^{30}}{0.005} \approx 1.72 \times 10^{12}\ \text{B/s} = 1.72\ \text{TB/s}.$$

This still does not tell us the exact DRAM traffic. Caches, write policies, ECC behavior, compression, and extra transactions can make physical movement differ from the useful-byte accounting.

Nsight therefore reports hardware counters for bytes and throughput at L1, L2, and device memory rather than asking developers to infer everything from tensor sizes.[3]

**Three numbers can all be different:** vendor peak bandwidth, useful effective bandwidth computed from the algorithm, and hardware-observed bytes/sec at a specific memory interface. State which one you mean.

This is also why cross-paper bandwidth numbers require caution. Different precisions, tensor shapes, cache states, clocks, ECC settings, and byte-counting conventions can make superficially similar GB/s figures incomparable.

# 31 缓存友好并不自动意味着速度快

对应原文第 35 页；点击查看相邻原文对照页

31 缓存友好并不自动意味着快速 只有当内存限制足以使改进发挥作用并且该技术不会在其他地方引入更大的成本时，改进局部性才有价值。

共享内存 分块（Tiling） 策略可以减少 HBM 流量，但会消耗足够的 共享内存（Shared Memory） 来减少块 驻留状态（Residency）。缓存持久布局可以提高 L2 命中率，但会增加索引或 同步（Synchronization） 开销。 融合内核（Fused Kernel） 可能会消除中间写入，但会创建降低有用并行性的 寄存器压力（Register Pressure）。

即使在内存系统内，一个优化级别也可能揭示另一个瓶颈。减少 HBM 流量可能会限制 L2 带宽。改善 L2 重用可能会暴露指令问题或 张量核心（Tensor Core） 利用率。 Nsight 的内存分析明确处理了几个不同的限制：繁忙的内存单元、单元之间的带宽以及内存指令管道的 吞吐量（Throughput）。 [3]

因此，正确的优化循环是因果关系：

1. 识别实际限制进度的接口或依赖项，

2. 改变时间表或布局以降低成本，

3.重新测量整个内核（Kernel），

4.检查接下来限制了什么。

更好的 缓存命中（Cache Hit） 速率是一个中间指标。目标是更低的 实际耗时（Wall-clock Time）、更高的 吞吐量（Throughput）、更低的尾部 延迟（Latency） 或其他显式应用程序指标。如果应用程序指标没有改善，则缓存优化可能无关紧要或被其他成本所抵消。

这是一门重复的系统课程。 占用率（Occupancy）、利用率、缓存命中（Cache Hit） 速率和带宽是诊断信号。它们本身都不是最终目标。

# 31 Cache-friendly does not automatically mean fast

Improving locality is valuable only if memory was limiting enough for the improvement to matter and if the technique does not introduce a larger cost elsewhere.

A shared-memory tiling strategy may reduce HBM traffic but consume enough shared memory to reduce block residency. A cache-persistent layout may improve L2 hit rate but add indexing or synchronization overhead. A fused kernel may eliminate an intermediate write yet create register pressure that lowers useful parallelism.

Even within the memory system, one optimized level can reveal another bottleneck. Reducing HBM traffic may move the limit to L2 bandwidth. Improving L2 reuse may expose instruction issue or Tensor Core utilization. Nsight's memory analysis explicitly treats several distinct limits: busy memory units, bandwidth between units, and the throughput of memory instruction pipelines.[3]

The correct optimization loop is therefore causal:

1. identify the interface or dependency actually limiting progress,
2. change the schedule or layout to reduce that cost,
3. remeasure the whole kernel,
4. check what became limiting next.

> A better cache hit rate is an intermediate metric. The goal is lower wall-clock time, higher throughput, lower tail latency, or another explicit application metric. If the application metric does not improve, the cache optimization may be irrelevant or offset by another cost.

This is a recurring systems lesson. Occupancy, utilization, cache hit rate, and bandwidth are diagnostic signals. None of them is the final objective by itself.

# 32 高带宽并不意味着低 延迟（Latency）

对应原文第 36 页；点击查看相邻原文对照页

32 高带宽并不意味着低 延迟（Latency） 带宽和 延迟（Latency） 描述了内存路径的不同属性。

内存接口可以同时支持许多传输，因此可以提供巨大的聚合带宽，而单个相关负载仍然需要大量时间才能完成。这些是路径的不同属性。

真实的 GPU 内存系统包含队列、存储体、缓存、聚结器和许多未完成的请求。观察到的依赖项 延迟（Latency） 会随着命中位置、争用和请求并发性而变化，因此一个永恒的“HBM 延迟（Latency）”数字通常会产生误导。

具有许多独立请求的 内核（Kernel） 可以填充内存 流水线（Pipeline） 并接近高带宽。指针追踪 内核（Kernel） 可能一次仅公开一个相关请求，从而阻止系统构建相同级别的并发性。两者都可以访问相同的 HBM。

这也解释了为什么延迟（Latency）隐藏和延迟（Latency）减少是不同的。 缓存命中（Cache Hit） 可以减少请求传输的距离。 线程束（Warp） 调度可能会将长请求与不相关的工作重叠。底层请求不一定会变得更快。在实际运行执行中可以看到更少的等待时间。

带宽询问聚合流量。 延迟（Latency） 询问依赖关系。AI 内核（Kernels） 通常需要足够的请求级和 线程束（Warp） 级并行性才能将 高带宽内存（High-bandwidth Memory） 系统转变为高应用程序 吞吐量（Throughput）。

AS03和AS04将为带宽上限添加定量模型（Models）。 AS09将回到驻留状态（Residency）和独立的线程束（Warps）如何帮助覆盖内存延迟（Latency）。

# 32 High bandwidth does not mean low latency

Bandwidth and latency describe different properties of a memory path.

A memory interface can sustain many transfers concurrently and therefore deliver enormous aggregate bandwidth while an individual dependent load still takes substantial time to complete. Those are different properties of the path.

Real GPU memory systems contain queues, banks, caches, coalescers, and many outstanding requests. The observed latency of a dependency changes with hit location, contention, and request concurrency, so one timeless “HBM latency” number is usually misleading.

A kernel with many independent requests can fill the memory pipeline and approach high bandwidth. A pointer-chasing kernel may expose only one dependent request at a time, preventing the system from building the same level of concurrency. Both can access the same HBM.

This also explains why latency hiding and latency reduction are different. A cache hit may reduce the distance a request travels. Warp scheduling may instead overlap a long request with unrelated work. The underlying request did not necessarily become faster. Less of its wait became visible in wall-clock execution.

**Bandwidth asks about aggregate flow. Latency asks about a dependency.** AI kernels often need enough request-level and warp-level parallelism to turn a high-bandwidth memory system into high application throughput.

AS03 and AS04 will add quantitative models for bandwidth ceilings. AS09 will return to how residency and independent warps help cover memory latency.

# 33 NVIDIA 和 AMD：相同的问题，不同的词汇

对应原文第 37 页；点击查看相邻原文对照页

33 NVIDIA 和AMD：相同的问题，不同的 词表（Vocabulary） 各个供应商共享的物理目标：保留靠近计算的有用数据并避免不必要的片外移动。名称和确切的组织有所不同。

概念 NVIDIA CUDA 示例 AMD CDNA 示例 线程私有 寄存器（Registers） 寄存器文件（Register File） VGPR / SGPR 文件 显式协作存储 共享内存（Shared Memory） LDS 靠近计算的第一个数据缓存 L1/TEX 组织每 CU L1 向量（Vector） 缓存 共享更深的缓存 L2 L2 加上特定于架构的最后一级缓存层次结构 大型设备内存（Device Memory） HBM 上MI300/MI350 系列上的 H100/H200/B200 HBM 执行组 线程束（Warp） 波前（Wavefront） AMD 的 CDNA 3 ISA 文档描述了每个计算单元 64 KiB 的 LDS 和 内存层次结构（Memory Hierarchy），每个 CU L1 缓存通过更深的共享缓存进行馈送。当前的 ROCm 架构文档列出了 MI300X 的每个 CU 32 KiB L1 向量（Vector） 缓存、32 MiB 聚合 L2 以及该产品系列的 256 MiB 末级缓存结构。[12][16]该表是一种翻译辅助工具，而不是一对一微架构等效性的声明。 NVIDIA 共享内存（Shared Memory） 和 AMD LDS 具有类似的程序员管理的协作角色，但它们的容量、存储体结构、指令和周围的缓存层次结构是特定于供应商的。

先讲原理，再讲厂商机制。“在芯片上部署可重复使用的图块”是可移植的推理。

“使用这个精确的共享内存剥离”或“假设这个缓存大小”是特定于体系结构的建议。

即使个别 GPU 代际（GPU Generations） 发生变化，这种分离也使手册保持有用。

# 33 NVIDIA and AMD: same problem, different vocabulary

The physical goal is shared across vendors: retain useful data close to compute and avoid unnecessary off-chip movement. The names and exact organization differ.

| Concept | NVIDIA CUDA example | AMD CDNA example |
|---|---|---|
| Thread-private registers | register file | VGPR / SGPR files |
| Explicit cooperative storage | shared memory | LDS |
| First data cache near compute | L1/TEX organization | per-CU L1 vector cache |
| Shared deeper cache | L2 | L2 plus architecture-specific last-level cache hierarchy |
| Large device memory | HBM on H100/H200/B200 | HBM on MI300/MI350 series |
| Execution group | warp | wavefront |

AMD's CDNA 3 ISA documentation describes 64 KiB of LDS per compute unit and a memory hierarchy with per-CU L1 caching feeding through deeper shared caches. Current ROCm architecture documentation lists 32 KiB L1 vector cache per CU for MI300X, 32 MiB aggregate L2, and a 256 MiB last-level cache structure for that product family.[12][16]

The table is a translation aid, not a claim of one-to-one microarchitectural equivalence. NVIDIA shared memory and AMD LDS serve similar programmer-managed cooperation roles, but their capacities, bank structures, instructions, and surrounding cache hierarchy are vendor-specific.

> Teach the principle first, then the vendor mechanism. "Stage a reusable tile on chip" is portable reasoning. "Use this exact shared-memory carveout" or "assume this cache size" is architecture-specific advice.

This separation keeps the handbook useful even as individual GPU generations change.

# 34 代架构改变了细节

对应原文第 38 页；点击查看相邻原文对照页

34 代架构改变了细节 内存层次结构概念老化缓慢，而容量和机制老化较快。当前的一些示例说明了为什么实施细节必须具有时间范围。

料斗 H100。 NVIDIA 记录了每个 流式多处理器（SM） 的 50 MB 二级缓存（L2 Cache） 和 256 KB 组合 L1/纹理/共享内存容量，其中最多 228 KB 配置为 共享内存（Shared Memory）。 H100 SXM 产品规格列出 80 GB HBM3 和 3.35 TB/s 内存带宽（Memory Bandwidth）。[5][7]布莱克威尔. NVIDIA 当前的调优指南为 B200 级计算能力 10.0 保留了 256 KB 的 L1/纹理/共享组合资源，并为 GB200 保留了 126 MB 的 L2 文档。 HGX B200 规格列出了 180 GB HBM3e 和每个 GPU 高达 8 TB/s。[6][9] AMD CDNA.MI300X 产品文档列出了 192 GB HBM3，速度为 5.3 TB/s。 MI355X 基于 CDNA 4，列出了 8 TB/s 的 288 GB HBM3E。 ROCm 文档还显示了 LDS 容量的 代际（Generation） 变化，MI350 系列部件比 MI300 系列部件每个 CU 暴露更多的 LDS。[10][11][16]这些数字都不会改变基本的层次结构课程。较大的 L2 可以保留较大的 工作集（Working Set）。更多 HBM 带宽提高了片外上限。更多 共享内存（Shared Memory） 可以启用更大的瓷砖或更深的管道。

但软件仍然需要可以利用这些资源的访问模式。

出版规则。在未来版本中重用此页面之前，根据当前架构文档重新检查容量、支持的分拆、缓存控制和产品带宽。不要仅仅因为此快照已过时就给常青章节添加时间戳。

# 34 Architecture generations change the details

Memory-hierarchy concepts age slowly, while capacities and mechanisms age quickly. A few current examples show why implementation details must be time-scoped.

**Hopper H100.** NVIDIA documents a 50 MB L2 cache and a 256 KB combined L1/texture/shared-memory capacity per SM, with up to 228 KB configured as shared memory. H100 SXM product specifications list 80 GB HBM3 and 3.35 TB/s memory bandwidth.[5][7]

**Blackwell.** NVIDIA's current tuning guide keeps a 256 KB combined L1/texture/shared resource for B200-class compute capability 10.0 and documents 126 MB of L2 for GB200. HGX B200 specifications list 180 GB HBM3e and up to 8 TB/s per GPU.[6][9]

**AMD CDNA.** MI300X product documentation lists 192 GB HBM3 at 5.3 TB/s. MI355X, based on CDNA 4, lists 288 GB HBM3E at 8 TB/s. ROCm documentation also shows generation changes in LDS capacity, with MI350-series parts exposing more LDS per CU than MI300-series parts.[10][11][16]

None of these numbers changes the basic hierarchy lesson. A larger L2 can retain a larger working set. More HBM bandwidth raises one off-chip ceiling. More shared memory can enable larger tiles or deeper pipelines. But software still needs access patterns that can exploit those resources.

**Publication rule.** Recheck capacities, supported carveouts, cache controls, and product bandwidth against current architecture documentation before reusing this page in a future edition. Do not time-stamp the evergreen chapters merely because this snapshot is dated.

# 35 种常见误解

对应原文第 39 页；点击查看相邻原文对照页

35 常见误解“如果 模型（Model） 适合 HBM，内存问题就解决了。”不。容量表示字节可以驻留在此处。运行时间仍然会受到流量、带宽或 延迟（Latency） 的限制。

“全局内存（Global Memory） 表示 HBM。”不。全局内存（Global Memory） 是一个地址空间。在到达 设备内存（Device Memory） 之前，全局负载可能由 L1 或 L2 提供服务。

“局部内存（Local Memory） 接近线程。”不。它的范围是本地的，但 CUDA 通过 设备内存（Device Memory） 支持它。[1]

“共享内存（Shared Memory）只是软件控制的L1。”太松了。它们可能共享最新 NVIDIA GPU 上的物理容量，但 共享内存（Shared Memory） 具有显式寻址、范围、同步（Synchronization） 和存储体行为。

“缓存命中（Cache Hit） 率高意味着 内核（Kernel） 高效。”未必。 内核（Kernel） 可能在其他地方受到限制，或者缓存可能是 推理服务（Serving） 总工作的一小部分。

“HBM 具有高带宽，因此单个加载速度很快。”带宽是总流量。 延迟（Latency） 是依赖延迟。高带宽并不能消除 延迟（Latency）。

“合并访问（Coalescing） 表示 threadi 必须读取 elementi。”不。我们的目标是用尽可能少的所需内存段覆盖 线程束（Warp） 的地址。排列的访问仍然可以合并。[1][2] “更多的重复利用总是更好。”重用机制可能会花费 寄存器（Registers）、共享内存（Shared Memory）、同步（Synchronization） 和 占用率（Occupancy）。节省的流量必须证明这些成本是合理的。

大多数记忆神话都来自于将多个级别或指标扁平化为一个单词。将地址空间、物理位置、容量、流量、带宽、延迟（Latency） 和重用分开。

# 35 Common misconceptions

**“If the model fits in HBM, memory is solved.”**
No. Capacity says the bytes can reside there. Runtime can still be limited by traffic, bandwidth, or latency.

**“Global memory means HBM.”**
No. Global memory is an address space. A global load may be served by L1 or L2 before reaching device memory.

**“Local memory is close to the thread.”**
No. It is local in scope, but CUDA backs it with device memory.[1]

**“Shared memory is just L1 that software controls.”**
Too loose. They may share physical capacity on recent NVIDIA GPUs, but shared memory has explicit addressing, scope, synchronization, and bank behavior.

**“A high cache hit rate means the kernel is efficient.”**
Not necessarily. The kernel may be limited elsewhere, or the cache may be serving a small fraction of total work.

**“HBM has high bandwidth, so individual loads are fast.”**
Bandwidth is aggregate flow. Latency is a dependency delay. High bandwidth does not eliminate latency.

**“Coalescing means thread $i$ must read element $i$.”**
No. The goal is to cover a warp’s addresses with as few required memory segments as practical. Permuted accesses can still coalesce.[1][2]

**“More reuse is always better.”**
Reuse mechanisms can cost registers, shared memory, synchronization, and occupancy. The saved traffic must justify those costs.

Most memory myths come from flattening several levels or metrics into one word. Keep address space, physical location, capacity, traffic, bandwidth, latency, and reuse separate.

# 36 实用记忆 理解框架（Mental Model）

对应原文第 40 页；点击查看相邻原文对照页

36 实用记忆 理解框架（Mental Model） 当AI 内核（Kernel） 出乎意料地慢时，请按此顺序遍历层次结构。

1. 识别活动数据。权重、激活（Activations）、KV 状态、临时图块、输出和元数据具有不同的大小和生命周期。

2. 选择边界。您是否在推理 寄存器（Register）/共享流量、L1/L2 流量、HBM 流量或主机设备流量？如果没有边界，“移动的字节数”就毫无意义。

3. 估计有用字节。在明确的假设下计算算法隐含的最小读写次数。

4.寻找重用。哪些 值（Values） 可以在 寄存器（Registers）、共享内存（Shared Memory） 或缓存被逐出之前重用？

5. 检查访问几何结构。线程束（Warp） 请求是否合并？步幅或布局是否会导致额外的扇区？

6. 将容量与速率分开。工作集（Working Set） 适合吗？如果是，所需流量是否仍会超出可用带宽或 延迟（Latency） 预算？

7. 分析实际路径。Nsight 的内存工作负载分析公开 L1、L2 和 DRAM 字节、命中率、请求和 吞吐量（Throughput）。使用计数器来测试 模型（Model），而不是将其视为证据。[3]

8. 重新测量端到端。仅当应用程序指标改善时，内存优化才会成功。

紧凑型 模型（Model）。数据从生命周期和重用模式开始。 内核（Kernel） 将该数据映射到 寄存器（Registers）、显式片上存储、硬件缓存和 设备内存（Device Memory）。 性能（Performance） 是根据每次传输在字节必须再次移动之前实现的有用工作量得出的。

模型（Model） 是下一本手册的基础。 AS03 将量化 算术强度（Arithmetic Intensity）。 AS04 将根据计算和带宽上限来安排工作负载。 AS07–AS09 将放大为 合并访问（Coalescing）、共享内存（Shared Memory）、寄存器（Registers） 和 占用率（Occupancy）。 AS11 将解释为什么融合可以删除整个层次结构的行程。

# 36 A practical memory mental model

When an AI kernel is unexpectedly slow, walk through the hierarchy in this order.

1. **Identify the active data.** Weights, activations, KV state, temporary tiles, outputs, and metadata have different sizes and lifetimes.
2. **Choose the boundary.** Are you reasoning about register/shared traffic, L1/L2 traffic, HBM traffic, or host-device traffic? "Bytes moved" is meaningless without a boundary.
3. **Estimate useful bytes.** Count the minimum reads and writes implied by the algorithm under explicit assumptions.
4. **Look for reuse.** Which values can be reused in registers, shared memory, or cache before they are evicted?
5. **Inspect access geometry.** Are warp requests coalesced? Are strides or layout causing extra sectors?
6. **Separate capacity from rate.** Does the working set fit? If yes, can the required traffic still exceed the available bandwidth or latency budget?
7. **Profile the actual path.** Nsight's Memory Workload Analysis exposes L1, L2, and DRAM bytes, hit rates, requests, and throughput. Use counters to test the model rather than treating it as proof.[3]
8. **Remeasure end to end.** A memory optimization is successful only when the application metric improves.

**Compact model.** Data starts with a lifetime and reuse pattern. The kernel maps that data onto registers, explicit on-chip storage, hardware caches, and device memory. Performance follows from how much useful work each transfer enables before the bytes have to move again.

That model is the foundation for the next handbooks. AS03 will quantify arithmetic intensity. AS04 will place workloads against compute and bandwidth ceilings. AS07–AS09 will zoom into coalescing, shared memory, registers, and occupancy. AS11 will explain why fusion can remove whole trips through the hierarchy.

# 参考文献和进一步阅读

对应原文第 41 页；点击查看相邻原文对照页

参考文献和进一步阅读

[1] 英伟达。 CUDA 编程指南，最新文档。内存空间、全局/本地/共享内存（Shared Memory）、设备内存访问、缓存控制和内存-吞吐量（Throughput） 指导。

https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] 英伟达。 CUDA C++ 最佳实践指南，最新文档。 合并访问（Coalescing），带宽计算、共享内存重用和内存优化。

https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3]英伟达。 Nsight 计算 性能剖析（Profiling） 指南。内存工作负载分析、L1/TEX、L2、DRAM、扇区、请求、命中率和 吞吐量（Throughput） 指标。

https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[4]英伟达。 CUDA 编程指南：计算能力。 寄存器（Register），计算能力的共享内存和缓存资源限制。

https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capability.html

[5]英伟达。 13.4. 料斗调整指南H100 内存系统、L2、共享内存/L1 资源和 HBM 详细信息。

https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6]英伟达。布莱克威尔调整指南 13.4。 B200/GB200 内存系统详细信息和共享内存/L1 资源行为。

https://docs.nvidia.com/cuda/blackwell-tuning-guide/

[7]英伟达。 H100 张量核心（Tensor Core） GPU 产品规格。 H100 SXM 为 80 GB HBM3 和 3.35 TB/s。

https://www.nvidia.com/en-us/data-center/h100/

[8]英伟达。 H200 张量核心（Tensor Core） GPU 产品规格。 H200 SXM 为 141 GB HBM3e 和 4.8 TB/s。

https://www.nvidia.com/en-us/data-center/h200/

[9]英伟达。 HGX AI 工厂参考架构：组件。 H100/H200/B200内存容量和带宽表。

https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b200/latest/components。

html

[10]AMD。 AMD Instinct MI300X 加速器。 192 GB HBM3、5.3 TB/s 和当前产品规格。

https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html

[11]AMD。 AMD Instinct MI355X GPU。 288 GB HBM3E、8 TB/s 和 CDNA 4 产品规格。

https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html

[12]AMD。 AMD Instinct MI300 CDNA 3 指令集架构/微架构文档。 LDS、L1、L2 和设备-内存层次结构（Memory Hierarchy）。

https://rocm.docs.amd.com/en/latest/reference/gpu-arch/索引（Index）.html

[13] Dao, T. 等人FlashAttention：具有 IO 感知功能的快速且内存高效的 精确注意力（Exact Attention）。 NeurIPS 2022。

https://arxiv.org/abs/2205.14135

[14] NVIDIA 弯刀。 CUDA 中的高效 通用矩阵乘法（GEMM）。分层 分块（Tiling）、共享内存和 寄存器（Register） 重用和流水线。

https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_通用矩阵乘法（GEMM）.html

[15] Williams, S.、Waterman, A.、Patterson, D.Roofline：针对多核架构的富有洞察力的可视化 性能（Performance） 模型（Model）。 ACM 通讯，2009 年。

https://escholarship.org/uc/item/5tz795vq

[16] AMD ROC。 AMD GPU 架构文档，包括 CDNA 3/CDNA 4 架构参考和 MI300/MI350 硬件描述。

https://rocm.docs.amd.com/en/latest/reference/gpu-arch/索引（Index）.html

# References and further reading

[1] NVIDIA. *CUDA Programming Guide*, current documentation. Memory spaces, global/local/shared memory, device-memory accesses, cache control, and memory-throughput guidance.
https://docs.nvidia.com/cuda/cuda-programming-guide/

[2] NVIDIA. *CUDA C++ Best Practices Guide*, current documentation. Coalescing, bandwidth calculations, shared-memory reuse, and memory optimization.
https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[3] NVIDIA. *Nsight Compute Profiling Guide*. Memory Workload Analysis, L1/TEX, L2, DRAM, sectors, requests, hit rates, and throughput metrics.
https://docs.nvidia.com/nsight-compute/ProfilingGuide/

[4] NVIDIA. *CUDA Programming Guide: Compute Capabilities*. Register, shared-memory, and cache resource limits by compute capability.
https://docs.nvidia.com/cuda/cuda-programming-guide/05-appendices/compute-capabilities.html

[5] NVIDIA. *Hopper Tuning Guide 13.4*. H100 memory system, L2, shared-memory/L1 resource, and HBM details.
https://docs.nvidia.com/cuda/hopper-tuning-guide/

[6] NVIDIA. *Blackwell Tuning Guide 13.4*. B200/GB200 memory-system details and shared-memory/L1 resource behavior.
https://docs.nvidia.com/cuda/blackwell-tuning-guide/

[7] NVIDIA. *H100 Tensor Core GPU product specifications*. 80 GB HBM3 and 3.35 TB/s for H100 SXM.
https://www.nvidia.com/en-us/data-center/h100/

[8] NVIDIA. *H200 Tensor Core GPU product specifications*. 141 GB HBM3e and 4.8 TB/s for H200 SXM.
https://www.nvidia.com/en-us/data-center/h200/

[9] NVIDIA. *HGX AI Factory Reference Architecture: Components*. H100/H200/B200 memory capacity and bandwidth table.
https://docs.nvidia.com/enterprise-reference-architectures/hgx-ai-factory-h100-h200-b200/latest/components.html

[10] AMD. *AMD Instinct MI300X Accelerators*. 192 GB HBM3, 5.3 TB/s, and current product specifications.
https://www.amd.com/en/products/accelerators/instinct/mi300/mi300x.html

[11] AMD. *AMD Instinct MI355X GPUs*. 288 GB HBM3E, 8 TB/s, and CDNA 4 product specifications.
https://www.amd.com/en/products/accelerators/instinct/mi350/mi355x.html

[12] AMD. *AMD Instinct MI300 CDNA 3 Instruction Set Architecture / microarchitecture documentation*. LDS, L1, L2, and device-memory hierarchy.
https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

[13] Dao, T. et al. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS 2022.
https://arxiv.org/abs/2205.14135

[14] NVIDIA CUTLASS. *Efficient GEMM in CUDA*. Hierarchical tiling, shared-memory and register reuse, and pipelining.
https://docs.nvidia.com/cutlass/latest/media/docs/cpp/efficient_gemm.html

[15] Williams, S., Waterman, A., Patterson, D. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. Communications of the ACM, 2009.
https://escholarship.org/uc/item/5tz795vq

[16] AMD ROCm. *AMD GPU architecture documentation*, including CDNA 3/CDNA 4 architecture references and MI300/MI350 hardware descriptions.
https://rocm.docs.amd.com/en/latest/reference/gpu-arch/index.html

# 参考图：针对哪个问题阅读什么内容

对应原文第 42 页；点击查看相邻原文对照页

参考图：针对哪个问题阅读什么内容 问题 最佳起始来源 CUDA 的内存空间及其范围是什么？ CUDA 编程指南 [1] 局部内存（Local Memory） 和 寄存器（Register） 溢出如何表现？ CUDA 编程指南 [1]、Nsight Compute [3] CUDA 示例中当前使用的 合并访问（Coalescing） 模型（Model） 是什么？

CUDA编程指南[1]、最佳实践[2]应该如何区分理论带宽和有效带宽？

CUDA 最佳实践 [2] 如何检查 L1、L2 和 DRAM 流量？ Nsight 计算 性能剖析（Profiling） 指南 [3] 当前每个 流式多处理器（SM） 寄存器（Register）/共享内存限制是多少？

计算能力 [4] H100 的内存系统有何特点？ Hopper 调校指南 [5]、H100 规格 [7] B200/GB200 的内存系统有何特点？ Blackwell 调整指南 [6]、HGX B200 规格 [9] 当前的 AMD 内存层次结构示例是什么？ AMD 架构文档 [12,16]，MI300X/MI355X 规格

为什么 通用矩阵乘法（GEMM） 通过 共享内存（Shared Memory） 和 寄存器（Registers） 暂存数据？

CUTLASS 高效 通用矩阵乘法（GEMM） [14] 为什么 FlashAttention 是一个内存层次结构示例？ FlashAttention论文[13] 算术强度（Arithmetic Intensity） 下一步将走向何方？ Roofline 论文 [15] 面向人工智能工程师、开发人员和认真学习者的简短技术手册

本页公式、数学记号、数值表格或代码，请见紧接的原文对照页。原始矢量内容未改写。

# Reference map: what to read for which question

| Question | Best starting source |
|---|---|
| What are CUDA's memory spaces and their scopes? | CUDA Programming Guide [1] |
| How do local memory and register spills behave? | CUDA Programming Guide [1], Nsight Compute [3] |
| What is the current coalescing model used in CUDA examples? | CUDA Programming Guide [1], Best Practices [2] |
| How should theoretical and effective bandwidth be distinguished? | CUDA Best Practices [2] |
| How do I inspect L1, L2, and DRAM traffic? | Nsight Compute Profiling Guide [3] |
| What are current per-SM register/shared-memory limits? | Compute Capabilities [4] |
| What is specific to H100's memory system? | Hopper Tuning Guide [5], H100 specs [7] |
| What is specific to B200/GB200's memory system? | Blackwell Tuning Guide [6], HGX B200 specs [9] |
| What are current AMD memory-hierarchy examples? | AMD architecture docs [12,16], MI300X/MI355X specs [10,11] |
| Why does GEMM stage data through shared memory and registers? | CUTLASS Efficient GEMM [14] |
| Why is FlashAttention a memory-hierarchy example? | FlashAttention paper [13] |
| Where does arithmetic intensity lead next? | Roofline paper [15] |